

Sistema de análisis digital de oscilaciones gestuales en pacientes con Parkinson mediante inteligencia artificial

Esp. Ing. Alex Damián Barria Cárdenas

Maestría en Inteligencia Artificial

Director: PhD. Ing. Fernando Bernuy (Universidad Nacional de Chile)

Jurados:

Mg. Ing. Agustín Baffo (FIUBA)

Mg. Ing. Clara Bureu (FIUBA)

Jurado 3 (pertenencia)

Ciudad de Buenos Aires, agosto de 2026

Resumen

Este trabajo presenta el desarrollo de un sistema de software que analiza grabaciones de video de personas que realizan ejercicios estandarizados de evaluación motora en pacientes con Parkinson. La solución produce medidas cuantitativas del movimiento para complementar la valoración clínica con información objetiva y trazable. La enfermedad de Parkinson altera el control motor y suele evaluarse en consulta mediante la observación de esos ejercicios, lo que puede introducir subjetividad y dificultar el seguimiento fino de la evolución.

Para su implementación, se aplicaron contenidos de la maestría en inteligencia artificial, en particular visión por computadora, aprendizaje automático con modelos preentrenados, ingeniería de software y operaciones de aprendizaje automático.

Agradecimientos

A mi familia, por el apoyo sostenido y la confianza depositada a lo largo de este recorrido formativo.

Agradezco a los docentes, autoridades y demás actores de la universidad y de la educación pública argentina que, en distintas etapas de mi formación, compartieron conocimientos, exigencia y compromiso con la enseñanza.

Reconozco también la orientación del PhD. Ing. Fernando Bernuy, director de este trabajo, y el aporte desinteresado de los distintos profesionales consultados en el ámbito clínico y técnico, cuya colaboración orientó y enriqueció aspectos centrales del presente trabajo.

Índice general

Resumen	I
1. Introducción general	1
1.1. Introducción general al tema	1
1.2. Contexto y motivación	3
1.3. Estado del arte	4
1.4. Objetivos del trabajo	6
1.4.1. Objetivo general	6
1.4.2. Objetivos específicos	6
2. Introducción específica	7
2.1. Frameworks y bibliotecas	7
2.2. Datasets y datos	8
2.3. Modelos preentrenados	10
2.4. Infraestructura	12
3. Diseño e implementación	13
3.1. Pipeline de datos	13
3.1.1. Ingesta de video y audio	13
3.1.2. Preprocesamiento de imagen	14
3.1.3. Calibración espacial	14
3.1.4. Fase estática y segmentación del ejercicio	15
3.2. Arquitectura de la solución analítica	16
3.2.1. Alcance del aprendizaje automático	17
3.2.2. Criterios de diseño y configuración	17
3.2.3. Contrato de datos entre etapas	18
3.2.4. Orquestación de la inferencia	18
3.2.5. Seguimiento temporal y señales derivadas	19
3.2.6. Métricas de movimiento implementadas	20
3.2.7. Agregación, exportación y trazabilidad	23
3.3. Integración e implementación del sistema	24
3.3.1. Diferencias entre ejecución por CLI y por API	24
3.3.2. Servicio HTTP y procesamiento en segundo plano	24
3.3.3. Manejo de errores y recuperación de sesión	25
3.3.4. Artefactos y consulta de resultados	25
3.3.5. Interfaz web	25
3.3.6. Despliegue local y en la nube	26
4. Ensayos y resultados	33
4.1. Metodología de evaluación del sistema	33
4.1.1. Protocolo de evaluación con profesionales de salud	33
4.1.2. Criterios de aceptación	35
4.2. Evaluación de la extracción de métricas	35

4.2.1.	Cumplimiento de funcionalidades planificadas	35
4.2.2.	Cobertura por tipo de ejercicio	36
4.2.3.	Resultados de la batería de pruebas automatizadas	37
4.2.4.	Evaluación del seguimiento longitudinal	38
4.3.	Análisis de errores	40
4.3.1.	Familias de degradación	40
4.3.2.	Casos ilustrativos	40
4.4.	Ensayos de integración	41
4.4.1.	Pruebas unitarias y de integración del pipeline	41
4.4.2.	Despliegue local y en la nube	41
4.4.3.	Síntesis de la verificación integral	42
5.	Conclusiones	43
5.1.	Logros alcanzados	43
5.2.	Conocimientos aplicados	44
5.3.	Trabajo futuro	44
	Bibliografía	45

Índice de figuras

1.1. Ejercicio de finger tapping utilizado en la evaluación clínica de movimientos finos de la mano.	2
1.2. Postura de brazos extendidos hacia el frente para la evaluación del temblor de mano.	2
1.3. Secuencia de la prueba nariz-dedo, desde la postura de brazos extendidos alternando el contacto con la nariz.	3
1.4. Ejercicio de apertura y cierre de manos, evaluado frecuentemente con los brazos estirados hacia el frente.	3
2.1. Ejemplo ilustrativo de los puntos de referencia entregados por el modelo <code>HandLandmarker</code> de <code>MediaPipe</code>	10
3.1. Detección del tablero <code>ChArUco</code> para estimación de relación milímetros por píxel.	15
3.2. Fase estática de la distancia pulgar-índice en una sesión de finger tapping.	16
3.3. Flujo funcional del sistema desde la entrada de video hasta reportes y visualizaciones.	19
3.4. Serie temporal de la distancia pulgar-índice durante una sesión completa.	23
3.5. Interfaz de carga de sesión.	27
3.6. Interfaz de visualización de resultados.	27
3.7. Arquitectura de despliegue local con <code>Docker Compose</code>	29
3.8. Arquitectura de despliegue del prototipo en Google Cloud Platform.	30
4.1. Evolución longitudinal del índice de fatiga en el ejercicio de tapping de dedos.	39
4.2. Evolución longitudinal de la velocidad pico de apertura en el ejercicio de apertura y cierre de manos.	39

Índice de tablas

1.1. Enfoques frecuentes para cuantificar el movimiento en Parkinson y contextos afines.	5
2.1. Bibliotecas y herramientas de terceros utilizadas en el desarrollo del sistema, agrupadas por capa funcional.	8
2.2. Caracterización resumida del conjunto de grabaciones de video utilizado en el desarrollo y la verificación del sistema.	9
2.3. Modelos preentrenados de terceros incorporados al sistema.	11
2.4. Servicios gestionados de GCP seleccionados para la prueba de concepto.	12
3.1. Etapas del pipeline de procesamiento de video y componentes asociados.	16
3.2. Artefactos generados por sesión y su consumo en la aplicación.	26
4.1. Síntesis cualitativa de la retroalimentación de profesionales de la salud.	35
4.2. Cumplimiento de funcionalidades planificadas frente al estado final de implementación.	36
4.3. Cobertura de implementación y pruebas automatizadas por tipo de ejercicio.	37
4.4. Inventario de la batería de pruebas automatizadas ejecutada al cierre del trabajo.	38

Capítulo 1

Introducción general

En este capítulo se contextualiza la enfermedad de Parkinson, la manera habitual de evaluar sus signos motores y se expone la motivación para incorporar análisis automático de video. También se sintetiza el estado del arte en visión por computadora aplicada a trastornos del movimiento y se enuncian los objetivos del trabajo.

1.1. Introducción general al tema

La enfermedad de Parkinson es un trastorno neurodegenerativo frecuente que compromete principalmente el sistema motor. Entre sus manifestaciones se cuentan la bradicinesia, la rigidez, el temblor en reposo y alteraciones de la marcha y el equilibrio. La gravedad y la evolución de estos signos condicionan la calidad de vida del paciente y las decisiones terapéuticas del profesional de la salud.

En la práctica neurológica se emplean escalas validadas que formalizan la exploración motora. Una referencia ampliamente adoptada es la parte motora de la escala unificada de la Sociedad Internacional de Parkinson y trastornos del movimiento (MDS por su sigla en inglés), conocida en la literatura como MDS-UPDRS parte III, que guía la observación de tareas como movimientos de dedos, postura y marcha [1]. Esas puntuaciones son útiles para comparar pacientes y momentos en el tiempo, pero dependen de la pericia del examinador y del contexto de la consulta, factores que introducen variabilidad entre observadores y entre sesiones.

En los últimos años creció el interés por registrar la exploración en video, ya sea en forma presencial o remota, y por apoyar la lectura clínica con algoritmos que midan trayectorias, ritmo y amplitud a partir de las imágenes [1, 2]. Esa línea de trabajo apunta a obtener descriptores cuantitativos reproducibles a partir de grabaciones convencionales, sin reemplazar al médico pero ofreciéndole un respaldo objetivo para el seguimiento.

Para situar visualmente el tipo de tareas que los profesionales solicitan durante la exploración motora, se presentan a continuación ejemplos de 4 ejercicios frecuentemente incluidos en protocolos como la parte motora de la escala MDS-UPDRS.

En la figura 1.1 se muestra un ejemplo de finger tapping: toques rítmicos entre el pulgar y el índice. En la imagen se observa la mano en la postura típica de la prueba, que permite al examinador valorar la amplitud, la regularidad y la fatiga del movimiento.

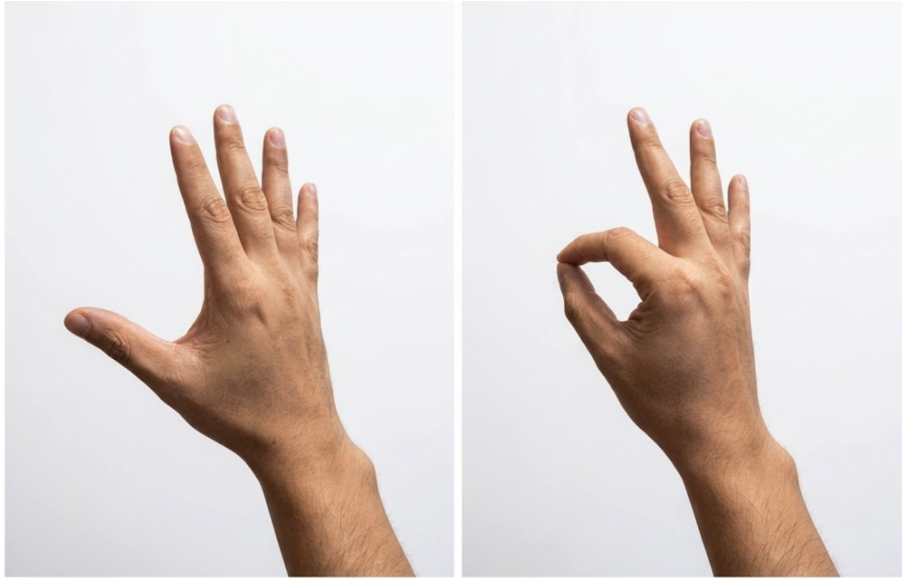


FIGURA 1.1. Ejercicio de finger tapping utilizado en la evaluación clínica de movimientos finos de la mano.

En la figura 1.2 se ilustra la evaluación del temblor de mano (hand tremor). En ella se observa a una persona de costado con los brazos extendidos hacia el frente, postura utilizada para evaluar los temblores posturales de las extremidades superiores.



FIGURA 1.2. Postura de brazos extendidos hacia el frente para la evaluación del temblor de mano.

En la figura 1.3 se detalla la prueba nariz-dedo (nose-finger). La secuencia consta de 4 imágenes: inicia con los brazos extendidos, continúa cuando el paciente se toca la nariz con el índice de una mano, vuelve a la postura inicial con ambos brazos extendidos y finaliza cuando se toca la nariz con el índice de la otra mano. Este ejercicio evalúa la precisión del alcance y la coordinación.



FIGURA 1.3. Secuencia de la prueba nariz-dedo, desde la postura de brazos extendidos alternando el contacto con la nariz.

Por último, en la figura 1.4 se exhibe el ejercicio de apertura y cierre de manos (open-close hands). En la imagen se aprecian las manos de una persona al realizar el ciclo de apertura y cierre. Este ejercicio suele evaluarse en una postura similar a la del temblor de mano, de pie y con los brazos estirados hacia el frente, para analizar la amplitud, la frecuencia y la velocidad de los movimientos.



FIGURA 1.4. Ejercicio de apertura y cierre de manos, evaluado frecuentemente con los brazos estirados hacia el frente.

Estas secuencias pueden registrarse en video para analizarlas de forma cuantitativa con herramientas de visión por computadora, como se desarrolla en los capítulos posteriores.

1.2. Contexto y motivación

La motivación del trabajo fue construir una herramienta que, a partir de videos de ejercicios motores, estimara la pose corporal del paciente y derivara métricas asociadas a oscilaciones y regularidad del movimiento. De este modo, se buscó reducir la dependencia exclusiva de la impresión visual humana para apreciar cambios leves o graduales.

La inteligencia artificial y la visión por computadora resultaron adecuadas porque permiten procesar cada fotograma de manera uniforme, conservar el detalle temporal entre cuadros consecutivos y transformar coordenadas articulares en magnitudes interpretables por el personal de salud. Se priorizó un enfoque basado en modelos preentrenados de estimación de pose, alineado con el alcance acordado para el proyecto, que explícitamente no incluyó entrenar redes neuronales desde cero en un conjunto propio de gran escala.

El trabajo culminó con un producto operativo que puede ejecutarse en entorno local y mediante contenedores, y además se desplegó en la nube sobre Google Cloud Platform (GCP) [3], de modo que el acceso al sistema no se limitó a una única máquina de desarrollo. La arquitectura y el procedimiento de despliegue quedaron documentados para facilitar extensiones posteriores, como validación prospectiva frente a lecturas de referencia o adaptaciones institucionales sujetas a marcos éticos y regulatorios propios de cada jurisdicción.

1.3. Estado del arte

La revisión sistemática de Pecoraro, Marsili, Espay, Bologna y di Biase sobre tecnologías de visión por computadora en trastornos del movimiento resume decenas de estudios entre 1984 y 2024 [2]. En ese conjunto de investigaciones predominan trabajos sobre la enfermedad de Parkinson y tareas de marcha. Para la estimación de pose sin marcadores físicos aparecen con frecuencia implementaciones basadas en OpenPose [4] y, en menor medida, otras soluciones mediante modelos propios o integraciones desarrolladas para cada estudio.

Los autores destacan que el análisis automático de video alcanzó en muchos informes exactitudes diagnósticas superiores al 80 % frente a etiquetas clínicas, con buena concordancia frente a acelerometría en temblor y distonía, aunque advierten heterogeneidad en condiciones de grabación y en el software empleado, lo que dificulta comparar resultados entre sitios.

En paralelo, el trabajo de Sibley, Girges, Hoque y Foltynie analiza retos metodológicos del análisis de video para cuantificar la severidad motora en Parkinson y el papel de la inteligencia artificial y el aprendizaje automático en ese escenario [1]. Concluyen que la evaluación por video es accesible y prometedora, pero que hacen falta muestras amplias y diversas y estándares de desempeño alineados con la práctica clínica antes de generalizar su uso.

Para la estimación de pose en tiempo casi real sobre hardware común, Google publicó el marco `MediaPipe`, que permite encadenar módulos de visión reutilizables y ajustar el consumo de recursos según el equipo disponible [5]. Dentro de ese ecosistema, el modelo de pose corporal ofrece coordenadas normalizadas de puntos anatómicos en dos y tres dimensiones y se utiliza en prototipos de salud y deporte por su equilibrio entre latencia y precisión aceptable para seguimiento de extremidades.

Entre las implementaciones recientes con fines clínicos y de investigación, VisionMD constituyó una plataforma de código abierto destinada al análisis automático de videos de tareas de la parte III de la escala MDS-UPDRS [6]. Mediante aprendizaje profundo localizó al sujeto, permitió marcar con precisión de fotograma el inicio y el fin de cada prueba y derivó variables cinemáticas como amplitud, velocidad y variabilidad a partir del seguimiento markerless de mano y cuerpo

con el marco `MediaPipe` [6, 5]. El procesamiento se ejecutó en equipos convencionales sin requerir hardware especializado ni envío obligatorio de los videos a servicios en la nube, en contraste con plataformas comerciales cerradas señaladas en la misma publicación.

En estudios piloto con pacientes con enfermedad de Parkinson, los autores reportaron diferencias significativas en métricas de tapping de dedos y otras tareas frente a estados de estimulación cerebral profunda encendida o apagada y frente a niveles de medicación, junto con concordancia inter-evaluador elevada en el uso de la herramienta [6]. Ese trabajo ilustró la viabilidad de cuantificar de forma objetiva pruebas ya incorporadas a la exploración motora estandarizada.

En diciembre del año 2025, como extensión de la plataforma VisionMD, Guarín presentó un método para cuantificar la amplitud y frecuencia del temblor postural de la mano directamente desde videos de teléfonos inteligentes, sin requerir referencias de calibración externas [7]. La propuesta combinó la estimación del diámetro del iris para establecer una escala métrica con modelos de estimación de pose 3D y seguimiento de manos, logrando una fuerte correlación con las puntuaciones clínicas tradicionales y sensibilidad a los efectos del tratamiento.

El prototipo documentado en esta memoria se inscribió en la misma línea de visión sin marcadores físicos basada en `MediaPipe Pose` y en el análisis de tareas motoras finas a partir de series de puntos clave, pero priorizó un flujo operativo orientado a servicios y despliegue mediante contenedores, como se describe en los capítulos siguientes.

En la tabla 1.1 se contrastan enfoques representativos del estado del arte en cuanto a insumos, salida típica y forma habitual de evaluación reportada en la literatura de revisión. No constituye un inventario exhaustivo de productos comerciales sino una guía conceptual para situar el trabajo desarrollado.

TABLA 1.1. Enfoques frecuentes para cuantificar el movimiento en Parkinson y contextos afines.

Enfoque	Datos de entrada	Salida típica	Evaluación habitual en la literatura
Escalas clínicas (MDS-UPDRS III y otras).	Observación en vivo o video asistido por humano.	Puntuaciones ordinales por ítem.	Consistencia inter-examinador, validez frente a estándares clínicos.
Sensores inerciales.	Aceleración o velocidad angular en segmentos corporales.	Series temporales, estimadores de frecuencia y amplitud.	Concordancia con video o con escala clínica.
OpenPose u otras redes de pose markerless.	Video monocular o multivista.	Coordenadas 2D o 3D de articulaciones.	Exactitud de clasificación o correlación con medidas de referencia [2].
MediaPipe Pose.	Video; modelos pre-entrenados integrados al marco.	Landmarks normalizados 2D/3D y continuidad temporal en un grafo de procesamiento integrado [5].	Idoneidad para tiempo casi real y bajo consumo en dispositivos habituales, frente a estimadores más pesados [5].

1.4. Objetivos del trabajo

Con el contexto clínico, la motivación del análisis automático de video y el panorama de la literatura ya expuestos, a continuación se enuncian el objetivo general y los objetivos específicos que guiaron el desarrollo del sistema.

1.4.1. Objetivo general

Diseñar e implementar un sistema de análisis digital de oscilaciones y desempeño motor en videos de ejercicios clínicos asociados a la enfermedad de Parkinson, apoyado en estimación automática de pose y en técnicas de procesamiento de señales e inteligencia artificial, que entregue métricas cuantitativas y visualizaciones comprensibles para apoyar el seguimiento objetivo del movimiento.

1.4.2. Objetivos específicos

1. Implementar un flujo de ingesta y preprocesamiento de video que alimente de manera estable el estimador de pose y módulos posteriores.
2. Integrar un modelo preentrenado de estimación de pose corporal y un seguimiento temporal de puntos relevantes para los ejercicios considerados.
3. Calcular métricas de amplitud, frecuencia, regularidad y estabilidad postural, u otras derivadas del mismo principio de análisis de series articulares, y agregarlas por sesión para su revisión estadística.
4. Exponer los resultados mediante una interfaz de servicios y una aplicación web que permitan cargar sesiones, inspeccionar gráficos y exportar reportes estructurados.
5. Verificar el comportamiento del software mediante pruebas unitarias y de integración sobre componentes críticos del flujo de procesamiento.

Los criterios de cumplimiento asociados a estos objetivos fueron la ejecución exitosa del flujo completo sobre videos de prueba disponibles para el proyecto, la coherencia de las métricas frente a escenarios controlados o simulados descritos en el capítulo de ensayos, y el paso de la batería de pruebas automatizadas definida en el repositorio del sistema.

Capítulo 2

Introducción específica

En este capítulo se describen las herramientas, bibliotecas, modelos preentrenados y conjuntos de datos provistos por terceros que sostuvieron el desarrollo del sistema. También se detallan el origen y las características de los videos utilizados y la infraestructura de ejecución contemplada para los ensayos y para el despliegue del prototipo.

2.1. Frameworks y bibliotecas

El sistema se construyó sobre tecnologías de código abierto. El núcleo del trabajo es un flujo de procesamiento de video y señales escrito en `Python 3.12` [8], sobre el que se montan un servicio HTTP, una capa de persistencia y una interfaz web. Esta separación permitió que la misma lógica de procesamiento se pudiera invocar desde una línea de comandos, un servicio HTTP o un contenedor sin alterar el núcleo del cálculo.

Para la lectura y manipulación de imagen se utilizó `OpenCV` [9], que cubrió la apertura de los archivos de video, la conversión entre espacios de color y las operaciones de redimensionado y muestreo cuadro a cuadro. La extracción del canal de audio asociado a las grabaciones se realizó con `moviepy`, y el análisis posterior de ese canal, en particular para la detección de los pulsos del metrónomo que estructuran los ejercicios, se apoyó en `librosa` [10], una biblioteca especializada en el procesamiento de señales de audio musicales y rítmicas.

El cálculo de las métricas se sostuvo en el conjunto de bibliotecas científicas habituales del ecosistema `Python`, integrado por `NumPy`, `SciPy` y `pandas` [11]. `NumPy` proveyó las operaciones vectoriales sobre las series temporales de los puntos detectados, `SciPy` aportó los estimadores espectrales y los algoritmos de detección de picos utilizados para el cálculo de frecuencia y de regularidad, y `pandas` se utilizó para organizar los datos por cuadro en estructuras tabulares que luego alimentan los pasos de agregación y exportación.

La capa de servicios se desarrolló con `FastAPI` [12], un framework para servicios HTTP en `Python` con soporte nativo para validación de datos a través de `Pydantic`, generación automática de documentación interactiva y eventos enviados desde el servidor (server-sent events) que se utilizaron para informar el progreso de cada análisis en tiempo casi real. La persistencia de pacientes, sesiones y resultados quedó a cargo de `PostgreSQL` [13], una base de datos relacional de uso amplio en aplicaciones de producción, accedida desde la aplicación mediante el object-relational mapper `SQLAlchemy` y administrada mediante `Alembic` para el versionado del esquema.

La interfaz de usuario se construyó como una aplicación web de página única en `TypeScript`, con un conjunto estándar de bibliotecas para componentes visuales, ruteo y gráficos interactivos. El detalle de esa capa se desarrolla en el capítulo de diseño e implementación.

En el plano de operación y empaquetado se adoptaron `Docker` [14] y `Docker Compose` para orquestar la base de datos, el servicio HTTP y un servidor web de archivos estáticos en una topología única reproducible en el entorno local y en el destino de despliegue. Las dependencias de `Python` se gestionaron con `uv`, mientras que la calidad del código se verificó mediante análisis estático y un conjunto de pruebas automatizadas con `pytest`.

En la tabla 2.1 se listan las bibliotecas y herramientas más relevantes para el procesamiento de datos y para el funcionamiento del sistema, agrupadas por su rol y con la motivación principal de su elección.

TABLA 2.1. Bibliotecas y herramientas de terceros utilizadas en el desarrollo del sistema, agrupadas por capa funcional.

Capa	Herramientas	Motivo de elección
Visión y video	MediaPipe, OpenCV	Modelos preentrenados de pose listos para uso y manipulación robusta de archivos de video.
Audio	librosa, moviepy	Extracción del canal de audio y detección de pulsos rítmicos para los ejercicios con metrónomo.
Numérica y métricas	NumPy, SciPy, pandas	Operaciones vectoriales, estimadores espectrales y manipulación tabular ampliamente probadas.
Servicios HTTP	FastAPI, Pydantic	Validación de datos integrada y soporte nativo para flujos de progreso por SSE.
Persistencia	PostgreSQL, SQLAlchemy, Alembic.	Base relacional madura, ORM y versionado declarativo del esquema.
Modelo de lenguaje	OpenAI Python SDK	Acceso a un modelo de lenguaje generativo para resúmenes interpretativos de las sesiones.
Operación	Docker, Docker Compose, uv	Despliegue equivalente en local y nube y dependencias reproducibles.

La elección de tecnologías favoreció componentes de uso difundido y bien documentados, lo que se alineó con los supuestos del trabajo sobre disponibilidad de bibliotecas de código abierto y contribuyó a mitigar el riesgo identificado en la planificación inicial respecto de posibles incompatibilidades en la integración de modelos de visión por computadora.

2.2. Datasets y datos

Las necesidades de datos del trabajo se concentraron en disponer de una cantidad de video suficiente para verificar la integración del flujo de procesamiento, evaluar la coherencia de las métricas en escenarios reproducibles y exponer la herramienta a movimientos representativos de los ejercicios contemplados.

No se incorporaron conjuntos de datos públicos. La conformación de un conjunto de grabaciones de pacientes con enfermedad de Parkinson exige un marco de

consentimiento informado, anonimización y resguardo bajo regulaciones de protección de datos de salud, condiciones que se inscriben en una etapa posterior a la de un prototipo de investigación. Esa restricción se reflejó desde la planificación inicial, que excluyó explícitamente del alcance la validación clínica formal y la integración con sistemas hospitalarios y dejó la evaluación funcional con especialistas como actividad opcional.

En el desarrollo se realizó además un análisis preliminar de los principales puntos de mejora y de criticidad para que el sistema pueda alinearse con normativas vigentes de protección de datos sanitarios, como la *Health Insurance Portability and Accountability Act* (HIPAA) [15] en su jurisdicción de origen. El resultado de ese análisis se desarrolla en los capítulos posteriores como trabajo futuro y propuestas de mejora.

Para sostener el avance del desarrollo dentro del alcance descripto, se trabajó con dos fuentes complementarias de video: grabaciones propias de personas sin patologías motoras que reprodujeron los ejercicios estandarizados y un conjunto reducido de grabaciones de pacientes obtenidas en condiciones controladas y bajo acuerdo de uso interno, que se mantuvieron por fuera del repositorio de código.

Esta estrategia resultó adecuada para los objetivos del trabajo porque el sistema se apoya en modelos preentrenados de estimación de pose, cuya capacidad de detectar puntos anatómicos no depende del diagnóstico de la persona registrada. Las grabaciones de personas sin patologías motoras resultaron suficientes para verificar la robustez del seguimiento de keypoints bajo distintos encuadres y condiciones de iluminación, mientras que las grabaciones de pacientes permitieron observar el comportamiento del sistema en presencia de los patrones motores característicos. La ampliación posterior del conjunto con material clínico anotado se identificó como una línea natural de continuidad y se desarrolla en el capítulo de conclusiones.

Las grabaciones se obtuvieron con la cámara posterior de un teléfono iPhone 15 Pro a 30 cuadros por segundo, en disposición fija sobre una superficie plana y con la región de interés (mano dominante o tronco superior, según el ejercicio) centrada en el cuadro. Durante los primeros segundos de cada grabación se ubicó en el campo visual un tablero de calibración ChArUco [16], que el sistema utiliza para estimar la conversión píxeles–milímetros sin requerir intervención manual del operador. En todas las fuentes del conjunto se registraron los mismos ejercicios: finger tapping, temblor de mano, prueba nariz–dedo y apertura y cierre de manos. En la tabla 2.2 se resumen las características de cada fuente de video y las condiciones de captura asociadas.

TABLA 2.2. Caracterización resumida del conjunto de grabaciones de video utilizado en el desarrollo y la verificación del sistema.

Fuente	Notas de captura
Grabaciones propias de personas sin patologías motoras.	Cámara fija, iluminación ambiente, tablero de calibración al inicio.
Grabaciones reducidas de pacientes.	Obtenidas bajo acuerdo de uso interno, almacenadas por fuera del repositorio.
Material de muestra anonimizado.	Incluido en el repositorio para pruebas de integración continua y para la incorporación de nuevos colaboradores.

El conjunto se administró bajo una clasificación de tres niveles definida en el repositorio del sistema: material de muestra anonimizado, grabaciones reales de pacientes y artefactos de salida del análisis, con políticas explícitas de almacenamiento y de exclusión del control de versiones para los dos últimos. Esa política se documentó internamente para evitar la difusión accidental de datos sensibles y se alinea con los requerimientos éticos enunciados en la planificación inicial.

2.3. Modelos preentrenados

Una de las decisiones de alcance establecidas al inicio del trabajo fue utilizar modelos preentrenados de uso difundido en lugar de desarrollar arquitecturas propias. Esta elección se fundamentó en el equilibrio entre el costo de obtener un conjunto etiquetado de gran tamaño y la disponibilidad de soluciones maduras para la estimación de pose de mano y de cuerpo completo, ya validadas en aplicaciones de salud, deporte y realidad aumentada.

Para la estimación de pose se utilizaron dos modelos del marco MediaPipe [5], distribuidos a través de la API de tareas (Tasks API) en su versión 0.10. El modelo `HandLandmarker` [17] entrega 21 puntos por mano detectada, organizados en falanges y articulaciones, y se utilizó en todos los ejercicios centrados en la mano. El modelo `PoseLandmarker` en su variante lite [18] entrega 33 puntos del cuerpo y se reservó para los ejercicios que requerían el contexto del torso y los brazos. Ambos modelos se descargaron desde el repositorio público de MediaPipe en formato float16 y se almacenaron localmente en una caché versionada de modelos.

En la figura 2.1 se muestra un ejemplo de los 21 puntos de referencia entregados por el modelo `HandLandmarker`, ubicados sobre la imagen de una mano en la postura típica del ejercicio de finger tapping.

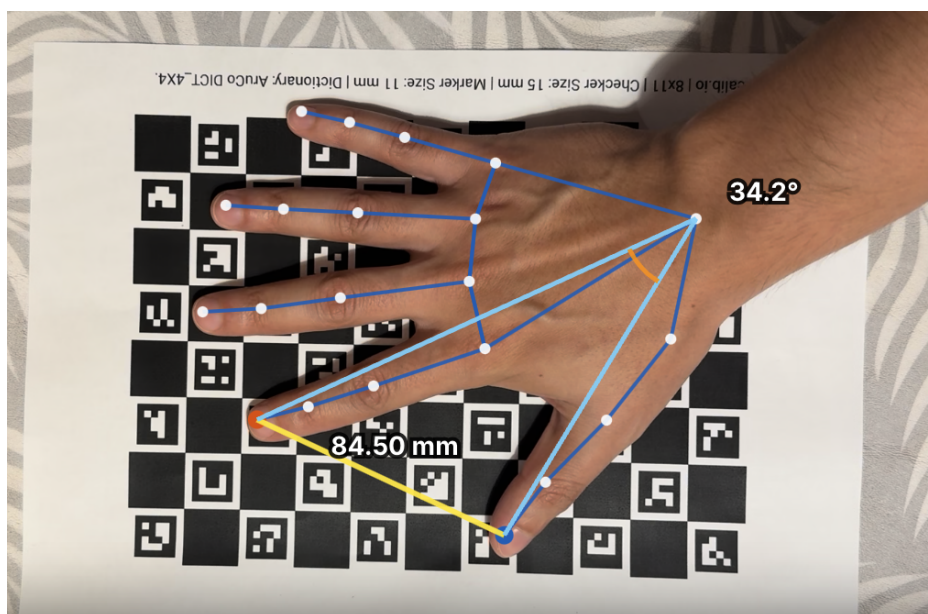


FIGURA 2.1. Ejemplo ilustrativo de los puntos de referencia entregados por el modelo `HandLandmarker` de MediaPipe.

En la imagen se observa la disposición regular de los puntos a lo largo de las falanges y las articulaciones, que permite reconstruir la geometría de la mano cuadro a cuadro y, a partir de ella, derivar las distancias y los ángulos sobre los que se calculan las métricas del sistema.

Para la generación automática de resúmenes interpretativos sobre los resultados de cada sesión y sobre la evolución longitudinal de un paciente se incorporó un modelo de lenguaje de gran tamaño (*Large Language Model*, LLM), accedido a través de la API de OpenAI [19].

El modelo elegido fue GPT-5.4-mini, una variante orientada a tareas conversacionales con costo y latencia acotados. El componente del sistema responsable de esta funcionalidad envía únicamente las métricas cuantitativas y los metadatos no identificatorios de la sesión, junto con instrucciones en formato de prompt estructurado. Luego, el sistema recibe un texto en lenguaje natural que el profesional de salud puede inspeccionar como apoyo a la lectura del reporte. El modelo nunca recibe identificadores del paciente, en línea con la política de manejo de datos del trabajo.

En la tabla 2.3 se listan los modelos preentrenados utilizados, con su origen, la tarea sobre la que operan y las características relevantes para el sistema.

TABLA 2.3. Modelos preentrenados de terceros incorporados al sistema.

Modelo	Tarea	Origen y datos de pre-entrenamiento	Uso en el sistema
HandLandmarker (MediaPipe 0.10)	Detección de 21 puntos por mano.	Imágenes anotadas de manos en distintas poses, entornos y condiciones de iluminación [17].	Ejercicios de finger tapping, temblor de mano y apertura y cierre de manos.
PoseLandmarker Lite (MediaPipe 0.10)	Detección de 33 puntos corporales.	Imágenes y video con anotaciones de pose corporal completa [18].	Prueba nariz-dedo (contexto de tronco y brazos).
GPT-5.4-mini (OpenAI)	Modelo de lenguaje generativo.	Conjunto general empleado por OpenAI para sus modelos de propósito general [19].	Resúmenes interpretativos por sesión y por evolución a partir de las métricas calculadas.

La integración de los modelos respetó los supuestos del trabajo sobre acceso a soluciones de código abierto y la decisión explícita de reutilizar modelos de visión ya consolidados. De ese modo se concentró el esfuerzo en el flujo de procesamiento, en la derivación de métricas y en la presentación clínica de los resultados, en lugar de en la construcción de un dataset de entrenamiento y de un protocolo de ajuste de redes neuronales. Otro aporte significativo del trabajo fue la operación e integración de los servicios necesarios para conformar una solución completa, que articula la infraestructura de ejecución, la interfaz de usuario, el servicio de aplicación y la persistencia en una arquitectura coherente y reproducible, descrita en el capítulo de diseño e implementación.

2.4. Infraestructura

Además de las bibliotecas de aplicación, el trabajo contempló tecnologías de infraestructura de terceros para empaquetar el sistema, ejecutarlo en entorno local y desplegarlo en la nube. En esta sección se caracterizan esas plataformas. La topología adoptada y el procedimiento de despliegue se desarrollan en el capítulo de diseño e implementación.

`Docker` [14] es una plataforma de contenedores que empaqueta una aplicación y sus dependencias en una imagen inmutable, ejecutable de forma aislada en sistemas operativos compatibles. `Docker Compose` permite declarar varios servicios interconectados, sus redes y volúmenes persistentes, y levantar la pila completa de manera coordinada. Es un patrón habitual para reproducir entornos de desarrollo sin instalaciones manuales.

Los modelos de estimación de pose son demandantes de memoria RAM pero, en el uso típico de `MediaPipe`, no requieren unidad de procesamiento gráfico dedicada. Debido a eso, la infraestructura prevista se orientó a cómputo en CPU con memoria ampliada, tanto en estaciones locales como en instancias gestionadas en la nube.

Para el despliegue en la nube se seleccionó Google Cloud Platform (GCP), ecosistema que admite ejecutar las mismas imágenes de contenedor definidas para el entorno local. En la tabla 2.4 se resumen los servicios gestionados seleccionados para la prueba de concepto de bajo costo en una única región.

TABLA 2.4. Servicios gestionados de GCP seleccionados para la prueba de concepto.

Servicio	Descripción y rol
Cloud Run	Contenedores HTTP serverless con escalado automático, memoria y tiempos de respuesta configurables para cargas largas.
Cloud SQL	PostgreSQL administrado, el Auth Proxy habilita conexión segura sin credenciales embebidas.
Cloud Storage	Objetos duraderos (videos, resultados, modelos) en buckets con prefijos lógicos.
Secret Manager	Secretos versionados inyectados en ejecución.
Cloud Build	Construcción de imágenes y despliegue automatizado hacia Cloud Run.

Las herramientas para el despliegue local (`Docker`, `Docker Compose`, `nginx`) y los servicios de GCP (`Cloud Run`, `Cloud SQL`, `Cloud Storage`, `Secret Manager`, `Cloud Build`) cubren cómputo, persistencia, archivos y credenciales de forma portable y gestionada. Configuraciones de mayor exigencia operativa o normativa quedaron fuera del alcance de este trabajo.

Capítulo 3

Diseño e implementación

En este capítulo se describen los criterios de diseño del sistema, el flujo de procesamiento de video desde la ingesta hasta la exportación de métricas, la arquitectura analítica, la orquestación de inferencia y el cálculo de métricas, y el desarrollo de la aplicación web con su despliegue. Las bibliotecas, modelos y servicios de terceros ya presentados en el capítulo anterior se asumen como base tecnológica y aquí se detalla cómo se organizan en módulos propios del trabajo.

3.1. Pipeline de datos

El núcleo del sistema se implementó como un paquete `pipeline` en `Python`, independiente de la capa `HTTP`, de modo que el mismo flujo puede ejecutarse desde la línea de comandos o desde el servicio en segundo plano de la API. El diseño priorizó la modularidad por etapas, la reproducibilidad de parámetros mediante configuración centralizada y la calibración automática de escala sin intervención manual del operador cuando el video incluye un tablero `ChArUco` al inicio de la grabación, tal como se documentó en la sección [2.2](#).

3.1.1. Ingesta de video y audio

La etapa de ingesta comienza con `VideoLoader`, que abre el archivo mediante `OpenCV`, extrae metadatos (resolución, cuadros por segundo, cantidad de fotogramas) y expone un iterador de fotogramas con muestreo configurable. En paralelo, `AudioExtractor` decodifica el canal de audio del contenedor de video a una señal monofónica en formato de punto flotante, requisito para detectar los pulsos del metrónomo que estructuran varios protocolos de grabación.

Sobre esa señal se aplica `detect_impulses`, que enfatiza el rango de frecuencias configurado (por defecto entre 400 y 7000 Hz), calcula la envolvente y localiza picos con un umbral de prominencia adaptativo basado en el rango intercuartílico de la envolvente. Los instantes detectados se almacenan como eventos de impulso con marca temporal. A partir de ellos, `estimate_metronome_bpm` estima los pulsos por minuto del metrónomo y una medida de confianza derivada de la variabilidad entre intervalos, información que luego se incorpora al resumen de sesión para contextualizar el ritmo esperado del ejercicio.

3.1.2. Preprocesamiento de imagen

Cada fotograma leído pasa por `FrameProcessor`, que convierte el espacio de color de BGR a RGB, redimensiona el ancho a un máximo de 640 píxeles cuando el video supera esa resolución y registra indicadores de calidad por cuadro. En el servicio HTTP el procesamiento se ejecuta en modo streaming: se lee un fotograma, se redimensiona, se estima la pose y se descarta el buffer, de modo que no se acumula en la memoria la secuencia completa. Para controlar el costo computacional en videos de 30 ó más cuadros por segundo, el algoritmo de la solución subsampla la secuencia a un máximo efectivo de 15 cuadros analizados por segundo y calcula el paso entre fotogramas como el cociente entero entre la tasa nominal y ese tope. La tasa efectiva resultante se utiliza en todas las métricas que dependen de la frecuencia de muestreo, en particular el análisis espectral del temblor.

3.1.3. Calibración espacial

Las distancias articulares se expresan preferentemente en milímetros. Para ello, el sistema implementó una estrategia de calibración con alternativas para el uso de un patrón físico impreso o la estimación biométrica sin marcadores.

Cuando el video incluye un tablero impreso al inicio de la grabación, se estima el factor de conversión píxeles a milímetros a partir de detecciones de un tablero ChArUco en una muestra de fotogramas. La geometría del tablero impreso (cuadrícula 5×7, lado de cuadrado 25 mm, marcadores ArUco `DICT_4X4_50`) coincide con la documentación interna del repositorio y con las condiciones de captura descritas en la sección 2.2. El muestreo de fotogramas para ChArUco se concentra en los primeros 12 s del video, con paso configurable, para privilegiar el intervalo en el que el tablero permanece visible. Cuando la detección de esquinas internas del tablero falla pero los marcadores ArUco individuales son visibles, se aplica un estimador alternativo basado en la longitud conocida del marcador, lo que incrementa la robustez ante oclusión parcial.

Para ejercicios en los que sostener un tablero impreso resulta impracticable, como en la evaluación del temblor de mano (`hand_tremor`), se implementó una calibración biométrica sin marcadores. Este método utiliza el ancho promedio del iris humano como referencia métrica conocida en el plano facial. Para trasladar esa escala desde el rostro hasta la mano que experimenta el temblor, se calcula la profundidad absoluta de ambas partes del cuerpo mediante la red neuronal MeTRAbs (del inglés *Metric Scale 3D Human Pose Estimation*) mencionada en la sección 2.3. Esta combinación permite ajustar el factor de conversión milímetros por píxel en la mano en función de la perspectiva de la cámara. Como medida de contingencia, el sistema conserva una calibración manual heredada (`legacy`) mediante parámetros de distancia directa en píxeles.

El resultado de la calibración se propaga al seguimiento y queda registrado en el resumen de sesión: método empleado (`charuco`, `markerless_iris_metrabs` o `legacy`), milímetros por píxel, tasa de detección, disponibilidad de la estimación de profundidad y parámetros geométricos. Esa trazabilidad permite al operador interpretar si las distancias en milímetros deben tomarse con reserva y habilita la auditoría visual del procedimiento.

En la figura 3.1 se puede observar un fotograma del video con el tablero detectado para estimar la relación milímetros por píxel. Cada uno de los recuadros verdes representa la detección de los marcadores del tablero. A partir de esa medición en píxeles y al conocer la dimensión real de los marcadores en milímetros, se puede obtener la relación milímetros por píxel buscada.

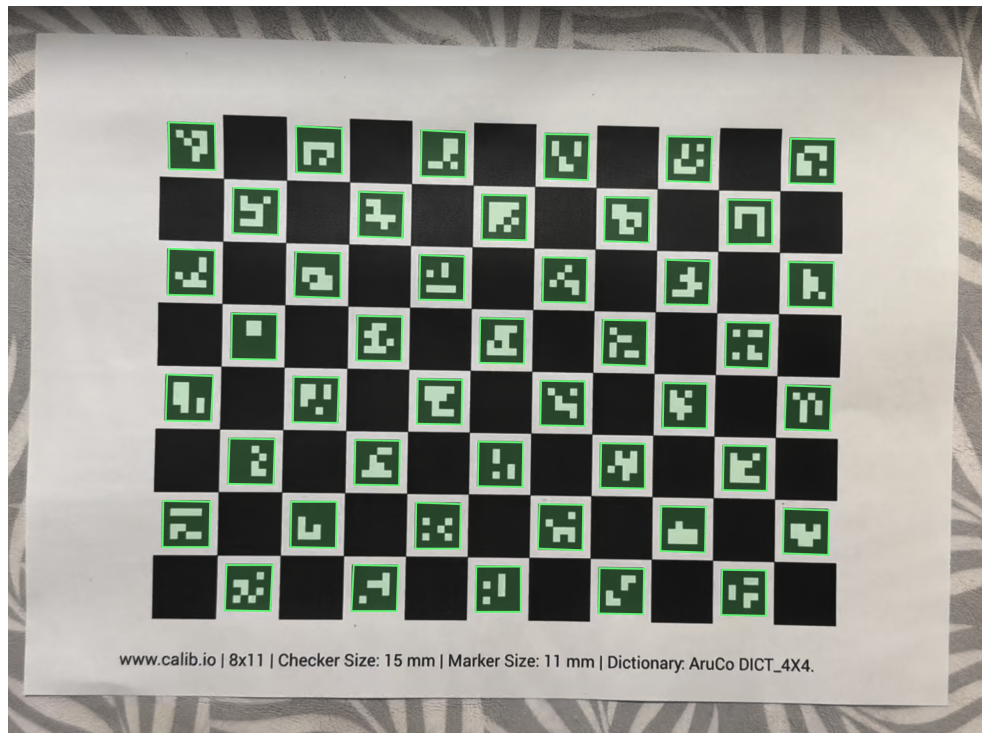


FIGURA 3.1. Detección del tablero ChArUco para estimación de relación milímetros por píxel.

3.1.4. Fase estática y segmentación del ejercicio

En el ejercicio de finger tapping, los primeros segundos de muchas grabaciones corresponden a la mano en reposo sobre el tablero de calibración. Para excluir ese tramo del cálculo de métricas dinámicas se implementó `detect_static_calibration_phase`, que calcula la desviación estándar móvil de la distancia pulgar-índice y marca como fase estática los intervalos en los que esa variabilidad permanece por debajo de 1,5 mm durante al menos 2 s. La función `slice_dynamic_after_static` recorta la serie temporal a partir del final de esa fase. Opcionalmente, en la API, el parámetro `auto_crop_taps` acota además la ventana al intervalo entre el primero y el último pico de tapping detectado en la señal, con un margen temporal de 0,1 s a cada lado.

En la figura 3.2 se observa una gráfica de la distancia pulgar-índice en función del tiempo, en la que se puede observar la fase estática y el inicio del segmento analizado para las métricas de tapping. Las líneas verticales rojas representan las detecciones de los sonidos del metrónomo que el paciente debe seguir con los movimientos del ejercicio a evaluar.

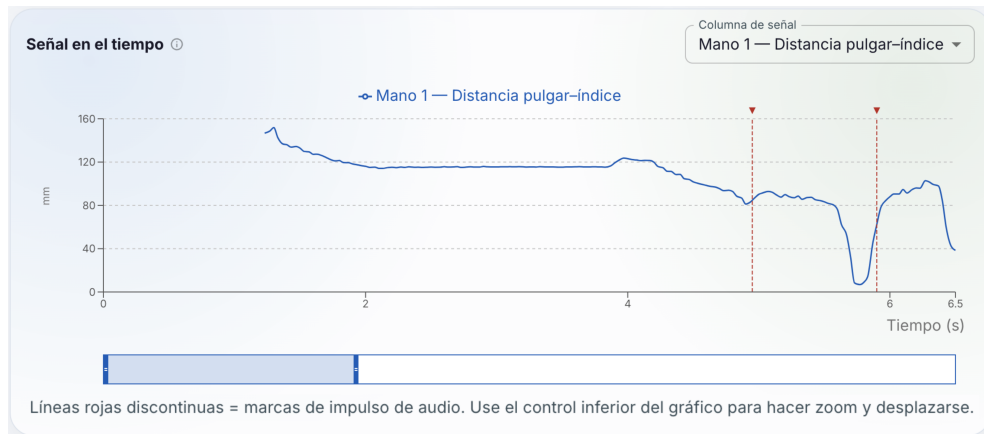


FIGURA 3.2. Fase estática de la distancia pulgar-índice en una sesión de finger tapping.

En la tabla 3.1 se resumen las etapas del flujo de procesamiento, los datos que recibe y produce cada una, y los módulos del repositorio que implementan la transformación.

TABLA 3.1. Etapas del pipeline de procesamiento de video y componentes asociados.

Etapas	Entrada	Salida	Módulo principal
Ingesta	Archivo de video.	Metadatos, audio mono, lista de impulsos.	VideoLoader, AudioExtractor.
Preprocesamiento	Fotogramas BGR.	Fotogramas RGB redimensionados.	FrameProcessor.
Calibración	Muestra de fotogramas iniciales.	Factor mm/px y objeto Calibration.	charuco, Calibration.
Estimación de pose	Fotograma procesado.	PoseResult por cuadro.	PoseEstimator.
Seguimiento	Secuencia de PoseResult.	DataFrame por fotograma.	KeypointTracker.
Segmentación	Serie temporal completa.	Subserie para métricas (tapping).	static_phase; recorte opcional.
Métricas	Serie de distancia pulgar-índice.	Lista de MetricResult.	pipeline.metrics.
Agregación y exportación	Tracking y métricas.	CSV, JSON, resumen de sesión.	SessionAggregator

Las etapas de ingesta, preprocesamiento y calibración corresponden al flujo documentado en esta sección. Las etapas de estimación de pose, seguimiento y segmentación se detallan en la sección 3.2.

3.2. Arquitectura de la solución analítica

Esta sección describe cómo se relaciona la inferencia con modelos ya presentados en el capítulo 2, la construcción de series temporales a partir de landmarks y las magnitudes cuantitativas derivadas para cada sesión. Se omite el detalle de

arquitecturas y pesos de terceros; el foco recae en la orquestación propia, el procesamiento de señales unidimensionales y la agregación orientada a la exploración clínica.

3.2.1. Alcance del aprendizaje automático

La arquitectura implementada se organizó como una cadena de módulos `Python` que orquestan inferencia con modelos preentrenados de visión, transforman las salidas en series temporales tabulares y calculan métricas en señales unidimensionales. En ese marco recayó el aporte principal del capítulo en materia de arquitectura: los contratos de datos entre etapas, el subsampleo temporal, la calibración, la agregación de sesión y el ajuste de parámetros de procesamiento de señal, umbrales de detección y reglas de segmentación documentadas en las subsecciones siguientes.

El alcance incluyó también un modelo de lenguaje de gran tamaño (LLM) como capa interpretativa sobre resultados ya cuantificados [19], orientado a resúmenes por sesión y por evolución longitudinal. El servicio `/api/insights` consume exclusivamente el `summary.json` de sesiones finalizadas y no reenvía video ni coordenadas de landmarks al proveedor externo. Las instrucciones se versionaron en plantillas YAML [20] (`session_summary.yaml` y `evolution_summary.yaml`) con rol de sistema y plantilla de usuario separados, de modo que el equipo pudiera iterar el texto sin modificar el código del enrutador.

Durante la implementación se evaluaron distintas variantes de prompt para acotar tono y extensión de la respuesta generada. Se fijó temperatura baja, límites de tokens de salida, secciones con encabezados predefinidos y prohibición explícita de formular diagnósticos, con recordatorio de que la interpretación final debe ser validada por el personal médico. Las variables interpoladas en cada solicitud se limitaron a metadatos de sesión, indicadores de calidad de la captura y tablas de métricas agregadas, sin nombres ni grabaciones del paciente, en línea con la política de manejo de datos del trabajo. El acceso al endpoint exige autenticación y verificación de que la sesión pertenece al profesional solicitante antes de invocar al modelo.

3.2.2. Criterios de diseño y configuración

La configuración global se centralizó en la clase `Settings`, que se carga con prefijo de variables de entorno `DAGO_` y valores por defecto versionados en el repositorio. De ese modo se evitó codificar rutas, umbrales de audio o geometría del tablero ChArUco dentro de la lógica. El tipo de ejercicio clínico (`finger_tapping`, `hand_tremor`, `nose_finger`, `open_close_hands`) se propagó a lo largo del pipeline para etiquetar métricas y seleccionar el modelo de pose adecuado.

Se adoptó además una política de clasificación de datos en tres niveles en el repositorio: material de muestra anonimizado apto para integración continua, grabaciones reales de pacientes excluidas del control de versiones y directorios de salida de análisis ignorados por el sistema de seguimiento de cambios. Esa separación se mantuvo coherente con los requerimientos éticos de la planificación inicial y condicionó que las pruebas automatizadas y los ejemplos de documentación apuntaran únicamente al nivel público, mientras las sesiones clínicas reales se procesaron en entornos locales o en la nube con almacenamiento privado.

3.2.3. Contrato de datos entre etapas

Para mantener la trazabilidad entre módulos se definieron esquemas de datos inmutables en cada frontera. `VideoMeta` y `FrameData` describen el origen temporal de cada fotograma; `AudioData` e `ImpulseEvent` encapsulan el audio y los pulsos detectados; `ProcessedFrame` añade las transformaciones de imagen; `PoseResult` agrupa las detecciones de mano y cuerpo por cuadro; y `MetricResult` estandariza la salida de cada algoritmo de métrica con nombre, unidad, validez y metadatos opcionales. El resumen final de sesión (`SessionSummary`) consolida métricas, retardos, parámetros de calibración, metrónimo y versión del pipeline en un único documento JSON serializable, apto para persistencia relacional y para los prompts del modelo de lenguaje.

Ese contrato permite sustituir o extender un módulo sin alterar los demás siempre que respete las interfaces acordadas. Por ejemplo, un futuro estimador de pose alternativo podría devolver el mismo `PoseResult` sin modificar el seguimiento ni las métricas aguas abajo.

3.2.4. Orquestación de la inferencia

El sistema se diseñó para no depender de un único modelo monolítico. La función `pose_model_for_exercise` selecciona el `landmarker` de `MediaPipe` según el tipo de ejercicio para extraer las señales de alta frecuencia: modo `HANDS` para `finger_tapping`, `hand_tremor` y `open_close_hands`, y modo `HOLISTIC` para `nose_finger`. La clase `PoseEstimator` instancia el modelo correspondiente mediante la API de tareas, con archivos `.task` en caché local. En paralelo y de forma específica para los fotogramas iniciales, el modelo `MeTRAbs` se orquesta para resolver la profundidad métrica y establecer el factor de escala global de la sesión.

Las coordenadas de profundidad (z) relativas que genera `MediaPipe` se almacenan en los esquemas internos pero no intervienen en distancias ni ángulos métricos, dado que la cuantificación operativa permanece en dos dimensiones escalada por la calibración inicial (`ChArUco` o biometría con `MeTRAbs`). En modo `HANDS` se retiene una sola mano dominante por fotograma; en modo `HOLISTIC` se registran además hombros, codos y muñecas para visualización.

La figura 3.3 muestra el flujo desde el video crudo hasta las interfaces de consulta. La estimación de pose emplea `HandLandmarker` para ejercicios de mano y `PoseLandmarker` lite para contexto corporal (sección 2.3). Además, se distingue el procesamiento analítico en CPU, la persistencia relacional y el acceso mediante API y web. Esta arquitectura condicionó las decisiones de integración detalladas en la sección 3.3.

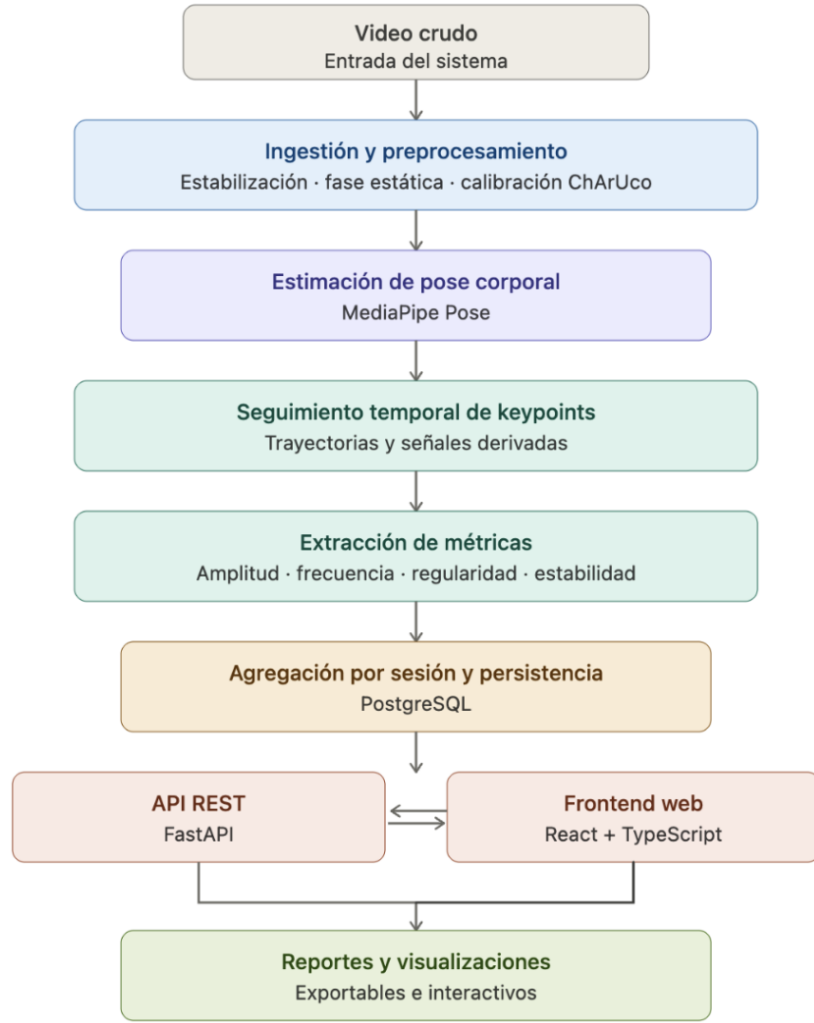


FIGURA 3.3. Flujo funcional del sistema desde la entrada de video hasta reportes y visualizaciones.

3.2.5. Seguimiento temporal y señales derivadas

KeypointTracker acumula, por cada fotograma analizado, un registro tabular común con índice temporal, coordenadas normalizadas de los 21 puntos de la mano dominante, distancias derivadas en milímetros cuando existe calibración y, en modo holístico, posiciones de nariz, hombros, codos y muñecas junto con los ángulos de abducción de brazo izquierdo y derecho (`pose_left_arm_angle`, `pose_right_arm_angle`). Las detecciones ausentes se registran como valores faltantes y se rellenan hacia adelante hasta un máximo de cinco fotogramas consecutivos para atenuar oclusiones breves. Esa capa de seguimiento es compartida por los cuatro tipos de ejercicio, pero la señal que alimenta las métricas clínicas se elige en función del movimiento evaluado.

En `finger_tapping` la señal operativa principal es la distancia pulgar-índice `hand0_thumb_index_dist`. Con factor de escala s milímetros por píxel y coordenadas normalizadas de pulgar e índice $\mathbf{p}_t$ y $\mathbf{q}_t$ en un fotograma de ancho W y alto H píxeles, se define la distancia d_t mediante la ecuación 3.1:

$$d_t = s \sqrt{W^2(p_{t,x} - q_{t,x})^2 + H^2(p_{t,y} - q_{t,y})^2}. \quad (3.1)$$

En `finger_tapping`, d_t sustentó los biomarcadores de tapping y la sincronía con el metrónomo. En `hand_tremor`, las trayectorias de los dedos índice, medio y anular de cada mano (asignadas a izquierda y derecha según la posición en imagen) alimentaron el cuantificador `TremorQuantifier`, que estimó amplitud y frecuencia del temblor postural por extremidad. En `open_close_hands`, la apertura de cada mano se expresó mediante las columnas `left_hand_openness_mm` y `right_hand_openness_mm`, obtenidas al reasignar los slots `hand0/hand1` de `MediaPipe` a la anatomía del paciente.

En `nose_finger` se normalizó la distancia yema del índice–nariz por el ancho de hombros, produciendo `left_finger_nose_norm` y `right_finger_nose_norm`. Los mínimos locales de esas series delimitaron contactos con la nariz; se validaron la postura inicial en cruz, la extensión del brazo contrario y el retorno al alcance. Como métricas secundarias se registraron la variabilidad del ángulo de abducción de cada brazo y la estabilidad postural del tronco mediante un proxy hombro–cadera.

3.2.6. Métricas de movimiento implementadas

El módulo de métricas seleccionó, según el tipo de ejercicio, la señal temporal y el conjunto de algoritmos aplicables. En `finger_tapping`, las funciones `oscillation_amplitude`, `tremor_frequency` y `movement_regularity` operaron sobre $\{d_t\}$ cuando hubo al menos cuatro muestras válidas; la frecuencia dominante se estimó por transformada rápida de Fourier en la banda definida en la ecuación 3.2:

$$f \in [1, 0, 12, 0] \text{ Hz}, \quad (3.2)$$

coherente con el rango de interés clínico para temblor y oscilaciones de extremidad superior [2], para lo cual se empleó la tasa efectiva tras el subsampling ($f_s = \text{fps}_{\text{video}}/\text{step}$).

En `finger_tapping`, tras recortar la fase estática y, de manera opcional, la ventana entre el primero y el último tap, se aplicaron sobre $\{d_t\}$ los biomarcadores de `compute_finger_tapping_metrics`: tasa de taps, intervalo intertap medio y su coeficiente de variación, decremento de amplitud entre picos, índice de fatiga, velocidades medias de apertura y cierre, y el promedio del ángulo en la muñeca (`thumb_index_wrist_angle_mean`). La detección de picos usó `scipy.signal.find_peaks` con prominencia proporcional al rango de la señal [1, 6]. `SessionAggregator` calculó además retardos entre impulsos del metrónomo y mínimos locales de d_t dentro de $\pm 0,35$ s, exportados en `delays.csv` y en el resumen como `movement_delays` (no como magnitud escalar en `metrics.csv`).

En `hand_tremor`, `TremorQuantifier` aplicó un filtro pasabanda Butterworth entre 2 y 10 Hz sobre la trayectoria promedio de los tres dedos y estimó, por mano, la amplitud pico a pico (`tremor_amplitude_left/right`) y la frecuencia principal por método de Welch (`tremor_frequency_left/right`). No se calcularon biomarcadores de tapping, regularidad de movimiento ni retardos con metrónomo.

En `nose_finger`, `compute_nose_finger_metrics` cuantificó repeticiones válidas, contactos por mano, ritmo alternado, sincronía con el metrónomo, amplitud de alcance normalizada, velocidades de aproximación y retorno, asimetrías

bilaterales, suavidad del gesto y oscilación durante el alcance. Como métricas secundarias se estimaron `postural_stability` y `arm_angle_variability_left/right` a partir del desplazamiento del centro de masa definido por hombros y caderas y del rango del ángulo de abducción de cada brazo.

En `open_close_hands`, `compute_open_close_hands_metrics` procesó las series bilaterales de apertura para obtener tasas de cierre por mano, intervalos entre cierres, decremento y fatiga de amplitud, velocidades en picos y valles, y asimetrías de amplitud, ritmo, velocidad y alineación con el metrónomo cuando el audio estaba disponible.

Cada magnitud se encapsuló en un `MetricResult` con unidad, validez y nota cuando los datos fueron insuficientes. En el listado siguiente se resume el catálogo por tipo de ejercicio.

■ Tapping de dedos:

- Amplitud de oscilación (`oscillation_amplitude`, mm): rango pico a pico de d_t .
- Frecuencia de temblor (`tremor_frequency`, Hz): pico espectral sobre d_t , banda definida según la ecuación 3.2.
- Regularidad del movimiento (`movement_regularity`, adim.): variabilidad y entropía muestral de d_t .
- Tasa de taps (`tap_rate`, 1/s): ritmo de taps asociado a bradicinesia.
- Intervalo intertap medio (`iti_mean`, s): tiempo medio entre taps consecutivos.
- Coeficiente de variación del ITI (`iti_cv`, adim.): consistencia del ritmo de tapping.
- Decremento de amplitud (`amplitude_decrement`, mm/s): pendiente de amplitud entre picos sucesivos.
- Índice de fatiga (`fatigue_index`, adim.): amplitud final respecto de la amplitud inicial en la ventana dinámica.
- Velocidad de apertura en picos (`peak_opening_velocity`, mm/s): velocidad media al abrir los dedos en los picos de d_t .
- Velocidad de cierre en valles (`peak_closing_velocity`, mm/s): velocidad media al cerrar los dedos en los valles.
- Ángulo medio en muñeca (`thumb_index_wrist_angle_mean`, °): apertura media pulgar-índice en la muñeca.
- Retardos audio-movimiento (`movement_delays`, s): desfase por impulso entre el metrónomo y el mínimo local de d_t en $\pm 0,35$ s (`delays.csv`); no se agregó como escalar único en `metrics.csv`.

■ Temblor de mano:

- Amplitud de temblor (`tremor_amplitude_left/right`, mm): rango pico a pico filtrado de la trayectoria promedio índice-medio-anular.
- Frecuencia de temblor (`tremor_frequency_left/right`, Hz): frecuencia principal por Welch en banda 2–10 Hz.

- Prueba nariz–dedo:
 - Repeticiones completadas (`repetitions_completed`, cont.): toques válidos con brazo contrario extendido y retorno a la postura en cruz.
 - Tasa de éxito (`touch_success_rate`, adim.): cociente entre repeticiones válidas y aproximaciones detectadas.
 - Conteo de contactos (`touch_count_left/right`, `failed_touches`, cont.): aproximaciones por mano y fallos de protocolo.
 - Ritmo de contactos (`touch_rate`, 1/s; `iti_mean`, s; `iti_cv`, adim.): cadencia alternada entre manos.
 - Sincronía con metrónomo (`on_beat_ratio`, adim.; `timing_error_mean`, `timing_error_abs_mean`, s; `timing_cv`, adim.): desfase respecto de los impulsos de audio.
 - Amplitud de alcance (`reach_amplitude_left/right`, adim.): rango normalizado (referencia extendida menos contacto, / hombros).
 - Velocidades de aproximación y retorno (`approach_velocity_left/right`, `return_velocity_left/right`, adim./s): picos de velocidad hacia la nariz y de vuelta.
 - Asimetrías (`amplitude_asymmetry`, `velocity_asymmetry`, `count_asymmetry`, `rhythm_asymmetry`, adim.): diferencias bilaterales de alcance, velocidad, conteo y ritmo.
 - Suavidad del gesto (`movement_smoothness_left/right`, adim.): regularidad de la trayectoria índice–nariz por mano.
 - Oscilación en el alcance (`tremor_frequency_left/right`, Hz): componente espectral baja del dedo índice durante el movimiento.
 - Estabilidad postural (`postural_stability`, adim./fotograma): balanceo del tronco, estimado como proxy hombro–cadera (métrica secundaria).
 - Variabilidad del ángulo de abducción (`arm_angle_variability_left/right`, °): rango pico a pico del ángulo de brazo (métrica secundaria; la media figura en los metadatos de la métrica).
- Apertura y cierre de manos:
 - Tasa de ciclos (`cycle_rate_left/right`, 1/s): eventos de cierre por segundo en cada mano.
 - Amplitud media de apertura (`mean_openness_amplitude_left/right`, mm): apertura media en los cierres detectados.
 - Intervalos entre cierres (`iti_mean_close_left/right`, s; `iti_cv_close_left/right`, adim.): ritmo y consistencia por mano.
 - Decremento de amplitud (`amplitude_decrement_left/right`, mm/s): pendiente de amplitud en cierres sucesivos.
 - Índice de fatiga (`fatigue_index_left/right`, adim.): cociente entre amplitud del último y primer tercio de cierres.

- Velocidad de apertura en picos (`peak_opening_velocity_left/right`, mm/s): velocidad media al abrir cada mano.
- Velocidad de cierre en valles (`peak_closing_velocity_left/right`, mm/s): velocidad media al cerrar cada mano.
- Asimetrías bilaterales (`amplitude_asymmetry`, `rhythm_asymmetry iti`, `velocity_asymmetry_open/close`, `fatigue_asymmetry`, adim.): comparación relativa entre manos.
- Alineación con metrónomo (`inter_hand_timing_lag`, s; `mean_close/open_delay_left/right`, s): retardos por mano y fase cuando hubo audio disponible.

En la figura 3.4 se ilustra, a modo de ejemplo para `finger_tapping`, la serie d_t con picos de tapping y las detecciones de los pulsos del metrónomo marcados con líneas verticales rojas.

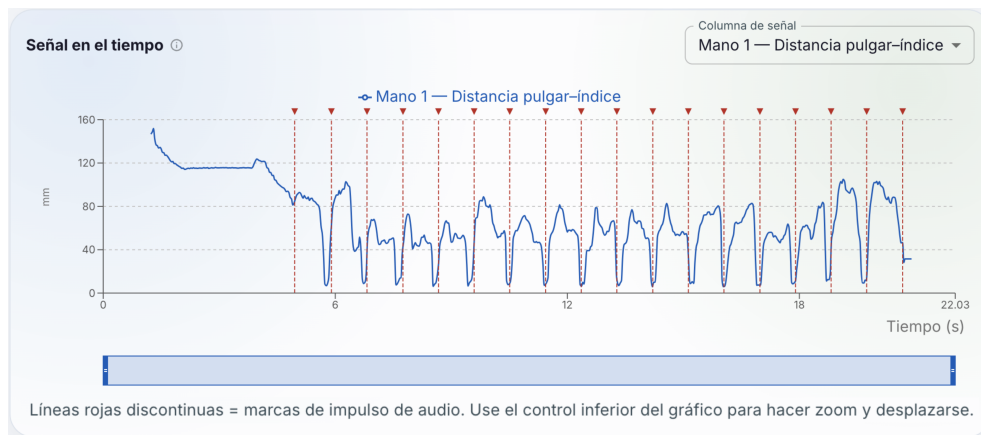


FIGURA 3.4. Serie temporal de la distancia pulgar-índice durante una sesión completa.

En la ejecución por línea de comandos se generan además gráficos `timeseries.png`, `metrics_summary.png` y `detection_overview.png` cuando la opción de guardado de figuras está activa. El worker de la API omite esos gráficos para reducir tiempo y uso de memoria en el contenedor y delega la visualización interactiva al frontend.

3.2.7. Agregación, exportación y trazabilidad

`SessionAggregator` recibe el `DataFrame` completo de seguimiento (incluida la fase estática cuando corresponde), la lista de métricas calculadas, los pulsos de audio y los metadatos de calibración. Produce un resumen con identificador de video, tipo de ejercicio, tasa de fotogramas nominal, conteo de fotogramas analizados, tabla de métricas y colección de retardos audio-movimiento. Los componentes `CsvWriter` y `JsonWriter` materializan ese contenido en disco con convenciones de nombres de columna estables, de modo que las pruebas de integración puedan verificar la existencia y el esquema de `tracking.csv`, `metrics.csv` y `summary.json` tras una ejecución de prueba del comando de sesión.

La coherencia entre definiciones matemáticas en código y descripción en la memoria se verificó en el capítulo 4 mediante pruebas unitarias sobre señales sintéticas con espectro y picos conocidos.

Cada objeto `MetricResult` incluye, además del valor escalar, un indicador de validez y un campo de nota textual cuando el algoritmo no puede estimar la magnitud (señal demasiado corta, menos de tres taps, espectro plano en la banda de temblor, entre otros casos). Esa decisión evitó presentar números engañosos en la interfaz y facilitó el diagnóstico de fallos de grabación (mano fuera de cuadro, oclusión prolongada, metrónomo inaudible). La versión del pipeline se registra en cada sesión para detectar incompatibilidades si en el futuro se modifican definiciones de métricas de forma no retrocompatible.

3.3. Integración e implementación del sistema

El prototipo operativo se estructuró en tres capas: el paquete `pipeline` como motor analítico, un servicio HTTP construido con `FastAPI` y una aplicación de página única en `React` servida mediante `nginx`. La persistencia de usuarios, pacientes y sesiones reside en `PostgreSQL`, con migraciones versionadas mediante `Alembic`, en línea con las herramientas citadas en la sección 2.1.

3.3.1. Diferencias entre ejecución por CLI y por API

Aunque ambos caminos comparten la lógica analítica, existieron diferencias operativas deliberadas en su diseño. La CLI procesa el video a la tasa de muestreo definida por el argumento `-step o`, por defecto, todos los fotogramas, mientras que el worker de la API impone el tope de 15 cuadros analizados por segundo y escribe el progreso en la cola SSE. La CLI genera figuras estáticas de control de calidad; la API prioriza la latencia y el pico de memoria del contenedor. La CLI acepta rutas de archivo locales; la API recibe bytes en memoria, los persiste en un archivo temporal y los elimina al finalizar; conserva únicamente la copia en el directorio de salida de la sesión. Esas variantes respondieron a usos distintos: experimentación offline e integración continua frente a carga interactiva desde el navegador.

3.3.2. Servicio HTTP y procesamiento en segundo plano

La aplicación `api.main` registra routers para autenticación (`/api/auth`), pacientes (`/api/patients`), sesiones de análisis (`/api/sessions`), comparación entre sesiones (`/api/compare`), generación de informes (`/api/sessions/.../report`) e interpretación asistida por modelo de lenguaje (`/api/insights`). Los esquemas de entrada y salida se validan con modelos `Pydantic`, lo que homogeneiza los tipos entre el cliente web y el servidor y reduce errores de formato en la carga multipart de videos.

El registro de usuarios y el inicio de sesión emiten tokens JWT que deben acompañar las peticiones subsiguientes. Los roles distinguen al menos cuentas de personal de salud y cuentas de portal de paciente con acceso de solo lectura a su propia evolución, de modo de separar la carga de nuevas sesiones de la consulta longitudinal sin exponer datos de terceros.

La carga de un video se realiza mediante `POST /api/sessions/analyse`, que crea un registro de sesión en estado pendiente y encola un trabajo en un `ThreadPoolExecutor` con capacidad limitada (dos hilos concurrentes y cola acotada) para evitar saturar la memoria del host. Si la cola supera el máximo configurado, el servicio responde con código HTTP 503 e informa al cliente que debe reintentar más tarde. Existe además un endpoint de análisis por lotes que procesa varios archivos en secuencia y mantiene el orden de envío; resulta útil para sesiones de laboratorio con múltiples repeticiones del mismo ejercicio.

Cada trabajo ejecuta `run_pipeline` en un hilo separado. Durante el procesamiento, los eventos de progreso (`ingestion`, `preprocessing`, `pose_estimation`, `metrics`, `aggregation`, `done` o `error`) se publican en una cola thread-safe asociada al identificador de sesión. El endpoint `GET /api/sessions/{id}/status` transmite esos eventos al navegador mediante eventos enviados desde el servidor (SSE), con autenticación por token en la cadena de consulta para compatibilidad con `EventSource`. Al finalizar, un co-routine asíncrono actualiza el campo `summary_json` y la ruta de salida en la tabla `AnalysisSession`.

3.3.3. Manejo de errores y recuperación de sesión

Si una excepción interrumpe el pipeline en el hilo de trabajo, el estado de la sesión pasa a `error`, se registra el mensaje en la base de datos y se emite un evento SSE de error al cliente. Los archivos temporales del video subido se eliminan en un bloque `finally` para no agotar el espacio en disco del contenedor. Cuando el proceso del servidor se reinicia y pierde la cola en memoria, el endpoint de estado puede consultar el registro persistido y devolver el estado terminal (`done` o `error`) tras un tiempo de espera acotado, de modo que el operador no quede indefinidamente suscrito a un análisis ya finalizado.

3.3.4. Artefactos y consulta de resultados

Por cada análisis exitoso se escriben en un directorio temporal (luego referenciado desde la base de datos) los archivos `tracking.csv` (serie temporal por fotograma), `metrics.csv` (tabla de métricas con validez y notas), `delays.csv` (retardos respecto del metrónomo), `summary.json` (resumen agregado con metadatos de calibración y metrónomo) y una copia `input.mp4` del video original para reproducción y superposición de marcadores ChArUco. El frontend recupera el resumen y el tracking mediante peticiones REST autenticadas y ofrece descarga de los artefactos y generación de informe en PDF mediante `ReportLab` en el servidor.

En la tabla 3.2 se relacionan esos productos con su uso en la interfaz y en los informes.

3.3.5. Interfaz web

La aplicación React define rutas para inicio de sesión y registro, carga de videos (`UploadPage`) con selección de ejercicio y parámetros de calibración, listado y detalle de pacientes, visualización de resultados por sesión (`ResultsPage`), exportación, comparación entre dos sesiones y un panel de progreso del paciente.

TABLA 3.2. Artefactos generados por sesión y su consumo en la aplicación.

Artefacto	Formato	Uso
<code>summary.json</code>	JSON	Resumen en pantalla de resultados, evolución longitudinal y prompts al LLM.
<code>tracking.csv</code>	CSV	Gráficos interactivos de series temporales y ángulos.
<code>metrics.csv</code>	CSV	Tabla de métricas exportable e informe PDF.
<code>delays.csv</code>	CSV	Análisis de sincronización con metrónomo (cuando aplica).
<code>input.mp4</code>	Video	Reproducción con superposición de landmarks y marcadores ChArUco.
Informe PDF	PDF	Exportación para revisión offline por el profesional.

El cliente HTTP centraliza las llamadas en `api/client.ts`, entre ellas la suscripción SSE que mapea las etapas del pipeline a mensajes legibles en la barra de progreso. La interfaz de carga de datos incorpora tres modalidades operativas: análisis individual, análisis por lotes (para procesar en segundo plano múltiples repeticiones de una sesión clínica) y modo de comparación para encolar dos videos de forma concurrente. En la figura 3.5 se muestra la interfaz de carga de sesión con la barra de progreso en tiempo real.

En la pantalla de resultados se combinan tablas de métricas con gráficos de series temporales contruidos con una biblioteca de gráficos declarativos, superposición opcional de landmarks sobre el video y solicitud de resúmenes interpretativos bajo demanda.

La vista de detalle de paciente integra un componente de evolución que filtra sesiones previas por tipo de ejercicio y representa la tendencia de biomarcadores seleccionados a lo largo del tiempo, en apoyo del seguimiento longitudinal previsto en los objetivos del trabajo.

La página de comparación envía las métricas de dos sesiones al servicio de insights para obtener un texto contrastivo, sin transmitir imágenes ni identificadores del paciente al proveedor del modelo de lenguaje.

En la figura 3.6 se presenta la vista de resultados de una sesión con gráficos de métricas, donde se omiten los datos identificables del paciente.

Los endpoints de interpretación envían al modelo de lenguaje únicamente métricas agregadas y metadatos no identificatorios, conforme a la política descrita en la sección 2.3, y devuelven texto en lenguaje natural como apoyo a la lectura del informe.

3.3.6. Despliegue local y en la nube

En entorno local, Docker Compose [14] orquestó cuatro servicios principales: base de datos PostgreSQL 16, API en Python 3.12 (imagen multi-etapa con dependencias instaladas mediante uv y límite de memoria de 12 GiB), frontend con nginx en el puerto 3000 (build estático de React y proxy de la API en el puerto 8000) y el sidecar MeTRAbs descrito en la sección 3.1. Este último se aisló en un contenedor propio de 10 GiB con interfaz FastAPI en el puerto interno 8081,

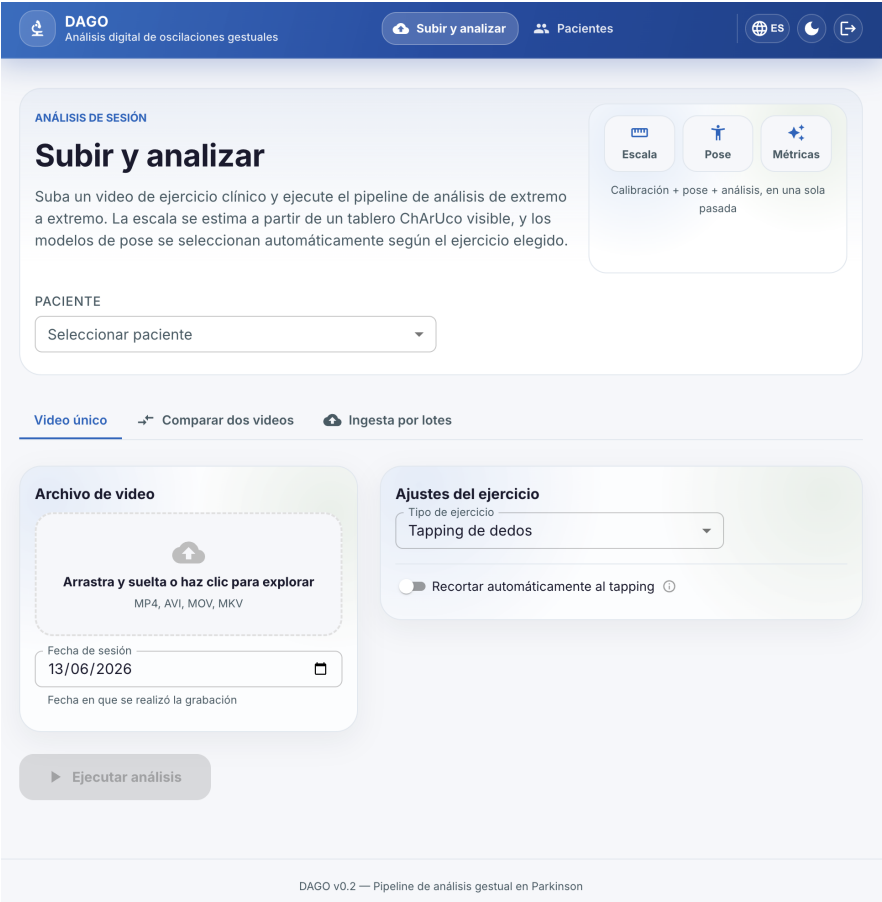


FIGURA 3.5. Interfaz de carga de sesión.

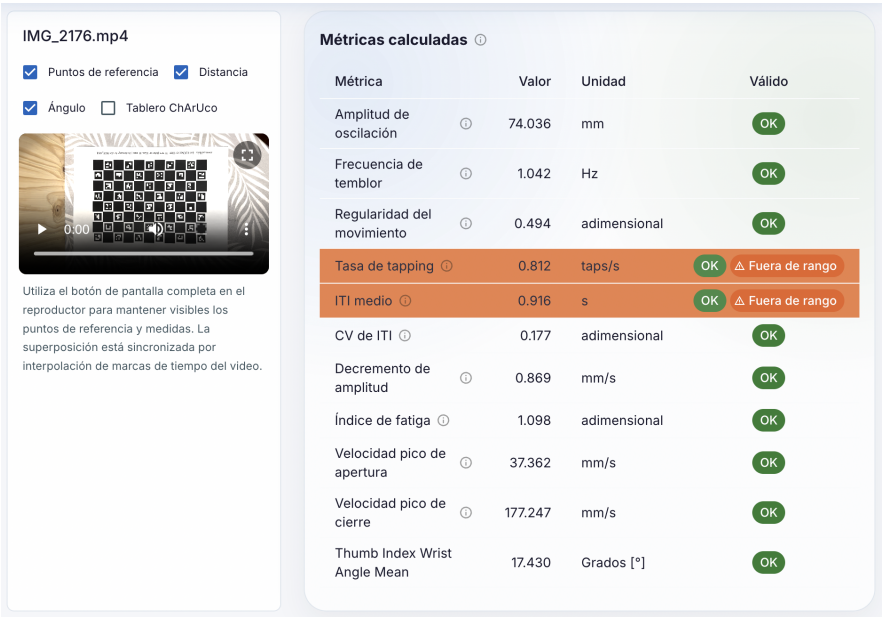


FIGURA 3.6. Interfaz de visualización de resultados.

de modo que la carga de TensorFlow no compitiera con MediaPipe en el servicio principal; la comunicación entre ambos contenedores se realizó por HTTP con autenticación por clave compartida.

Un perfil opcional permitió además ejecutar el mismo contenedor de la API en modo batch desde la línea de comandos. Los modelos de visión, los videos de muestra y el directorio de salida se montaron como volúmenes para facilitar la actualización sin reconstruir la imagen. La figura 3.7 resume la topología local.

En el diagrama local se observa la separación entre el acceso web, el procesamiento analítico y la base de datos, con MeTRAbs como servicio independiente. El punto de entrada del contenedor de la API ejecutó migraciones de esquema con Alembic antes de levantar uvicorn, de modo que un despliegue nuevo aplicara automáticamente las revisiones pendientes. El frontend se construyó en una etapa previa de la imagen (Node.js para compilar TypeScript) y el runtime de inferencia se fijó en linux/amd64 por compatibilidad con MediaPipe. La configuración del proxy desactivó el buffering en las rutas de la API y extendió el tiempo de lectura a 3600 s, requisito para transmitir el progreso del análisis mediante eventos enviados desde el servidor (SSE).

Para la prueba de concepto en la nube se adaptó el diseño a los servicios gestionados de GCP enumerados en la tabla 2.4, con el objetivo de reducir costos operativos y simplificar el despliegue inicial, sin replicar de forma idéntica la topología local. Un único servicio en Cloud Run concentró la API, el pipeline de análisis y la interfaz web empaquetada en el mismo contenedor; no se desplegó un proxy nginx separado ni el sidecar MeTRAbs. Cloud SQL alojó PostgreSQL 16 con conexión por socket Unix; Cloud Storage persistió videos y artefactos de sesión; Secret Manager inyectó las credenciales de base de datos y la clave de firma de tokens; Cloud Build construyó la imagen y la publicó en Artifact Registry.

Las migraciones de esquema se ejecutaron mediante un trabajo dedicado en Cloud Run Jobs, separado del arranque del servicio HTTP. La infraestructura se provisionó con Terraform en el entorno de prueba, con 4 vCPU, 8 GiB de memoria, concurrencia 1, tiempo máximo de 3600 s y CPU activa de forma continua para sostener hilos de procesamiento en segundo plano. La figura 3.8 resume las relaciones entre el usuario, el cómputo serverless y los servicios de datos y secretos.

En el diagrama de la nube se aprecia que el procesamiento de video y la inferencia con MediaPipe residen en el mismo servicio de cómputo que expone la API REST y sirve la interfaz web, mientras que la persistencia estructurada y los objetos de gran tamaño se externalizan a servicios administrados. Esa separación facilitó desplegar el prototipo sin duplicar el almacenamiento de videos en cada instancia, aunque el estado de progreso SSE permanece en memoria de proceso y el escalado horizontal quedó acotado a una sola réplica; ambas son limitaciones conocidas que se abordarán en el capítulo de conclusiones.

Con la arquitectura descrita quedó cerrado el ciclo de diseño e implementación: desde la captura estandarizada descrita en el capítulo anterior hasta la consulta web de métricas y exportación de informes. En el capítulo 4 se presentará el plan de evaluación y los resultados obtenidos al ejecutar este sistema sobre el conjunto de videos disponible para el trabajo, además de documentar la verificación

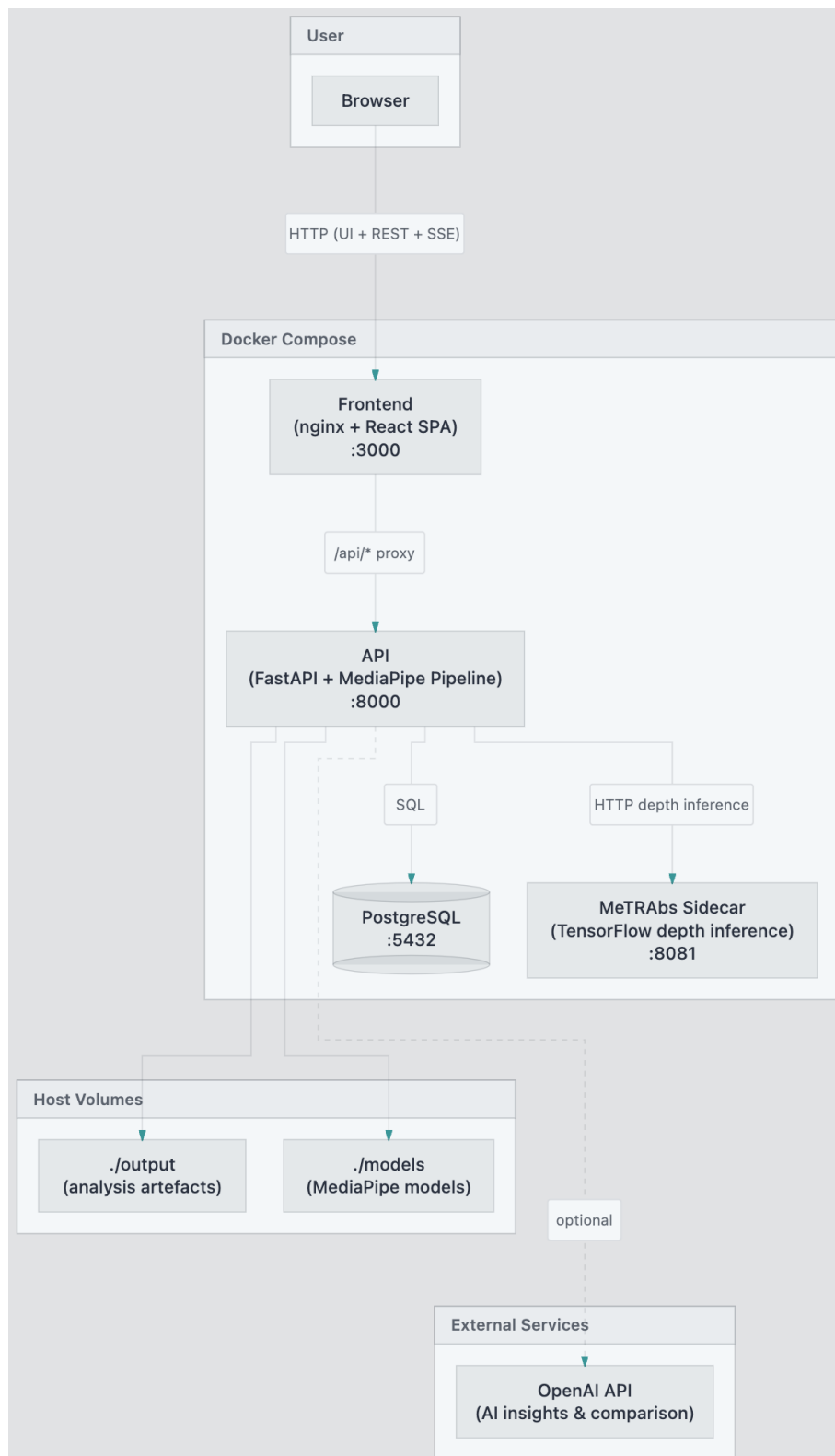


FIGURA 3.7. Arquitectura de despliegue local con Docker Compose.

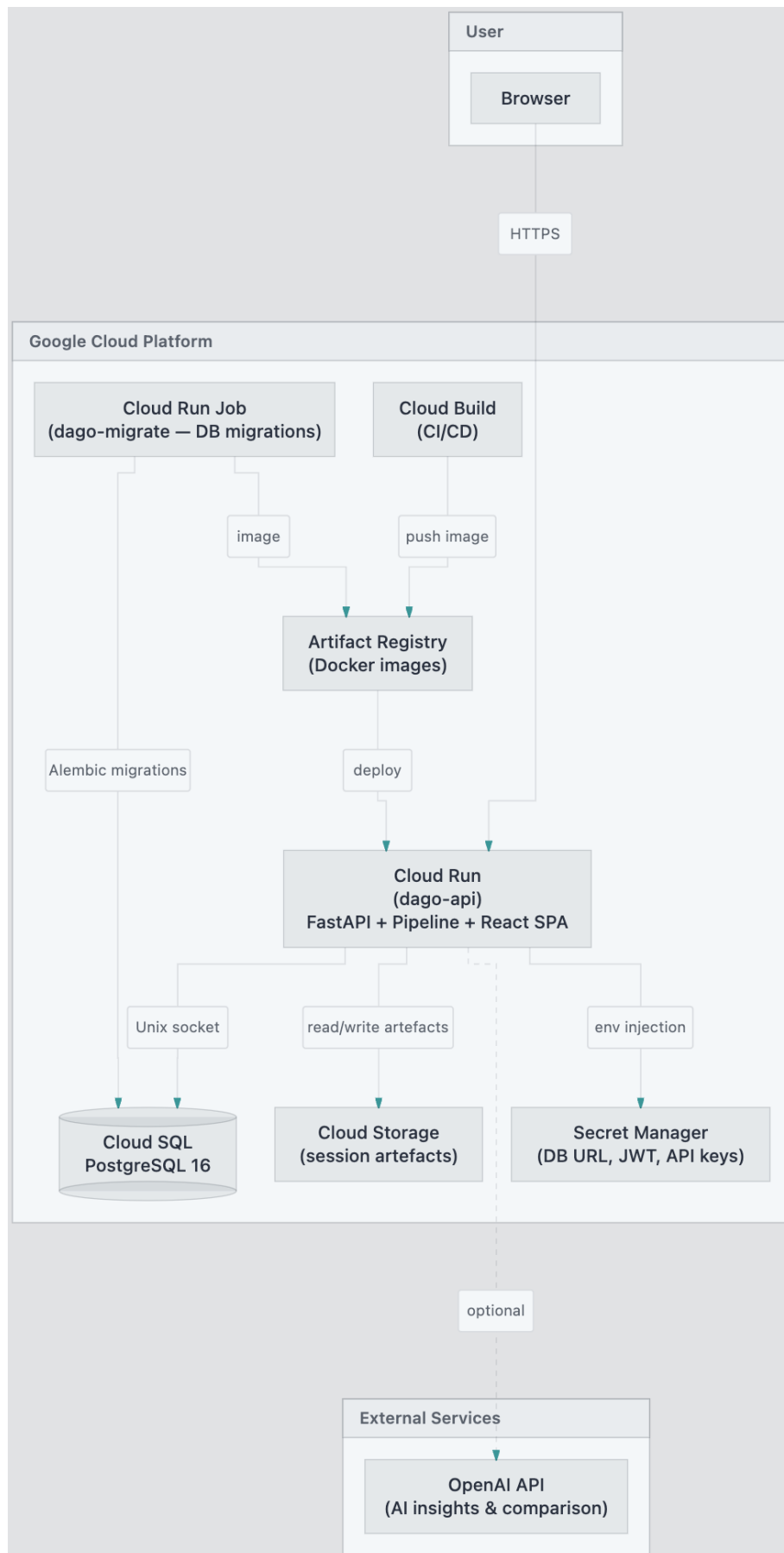


FIGURA 3.8. Arquitectura de despliegue del prototipo en Google Cloud Platform.

automatizada del desarrollo, las pruebas unitarias y la integración de extremo a extremo del pipeline.

Capítulo 4

Ensayos y resultados

En este capítulo se presenta la metodología empleada para evaluar la efectividad de la implementación del sistema desarrollado, en coherencia con los objetivos específicos del trabajo. El foco recae en el cumplimiento funcional de las capacidades planificadas, la cobertura automatizada por tipo de ejercicio, las pruebas de integración del software y la retroalimentación de profesionales de la salud.

4.1. Metodología de evaluación del sistema

Dado que el objetivo central del trabajo radica en automatizar la cuantificación de oscilaciones y desempeño motor a partir de video clínico, la evaluación se orientó a verificar que el sistema desarrollado ejecutara de forma reproducible el flujo planificado: ingesta de video, estimación de pose, seguimiento temporal, cálculo de métricas por tipo de ejercicio, agregación de sesión, exportación estructurada y exposición mediante servicios web e interfaz gráfica.

La estrategia de verificación se documentó en el repositorio del sistema y priorizó tres niveles de evidencia. En el nivel más granular, se aplicaron pruebas unitarias sobre secuencias sintéticas o controladas para validar el comportamiento matemático de cada módulo. En un nivel intermedio, se ejecutaron pruebas de integración que recorren el pipeline completo sobre material de video cuando el entorno lo permitía. En el nivel de despliegue, se verificó manualmente la operación del sistema empaquetado en contenedores locales y en una prueba de concepto en la nube.

Se adoptaron tres ejes complementarios de evaluación funcional. En primer lugar, se contrastó el inventario de funcionalidades priorizadas al inicio del trabajo con el estado final del código. En segundo lugar, se ejecutó la batería de pruebas automatizadas del repositorio y se registró el número de casos aprobados, omitidos y fallidos. En tercer lugar, se incorporó retroalimentación cualitativa de profesionales de la salud durante iteraciones sucesivas de la interfaz web.

4.1.1. Protocolo de evaluación con profesionales de salud

La instancia cualitativa se concibió como evaluación formativa de usabilidad y pertinencia clínica, no como estudio de validación diagnóstica. Se realizaron tres rondas de demostración durante el desarrollo de la interfaz: una primera con prototipo funcional de carga y resultados básicos, una segunda con gráficos de series temporales y tablas de biomarcadores, y una tercera con exportación de informes en PDF y vistas orientadas al seguimiento longitudinal.

En cada ronda participaron profesionales con perfiles complementarios. Por un lado, se consultó a neurólogos con experiencia en investigación de trastornos del movimiento, quienes aportaron criterio sobre la relevancia de las métricas agregadas para el seguimiento de pacientes con enfermedad de Parkinson y sobre la interpretabilidad de los gráficos en un contexto de consulta clínica o de protocolo de investigación. Por otro lado, se incorporó la opinión de kinesiólogos y profesionales de rehabilitación habituados a la evaluación motora en pacientes neurológicos, con énfasis en la legibilidad de las curvas de movimiento, la utilidad de los informes exportables para documentar la evolución entre sesiones y la adecuación del flujo de carga para un entorno de uso no técnico.

En total intervinieron cuatro profesionales a lo largo del ciclo de iteraciones, sin que una misma persona participara necesariamente en las tres rondas. En cada sesión se presentó el flujo completo: carga de video con selección del tipo de ejercicio, visualización del progreso del análisis, pantalla de resultados con gráficos y tabla de métricas, exportación de informe y, en la última ronda, la vista de evolución longitudinal y el modo de comparación entre dos sesiones del mismo paciente. Los participantes completaron una guía semiestructurada con cinco ítems en escala Likert y un espacio abierto para observaciones.

La valoración global fue positiva. Los profesionales coincidieron en que la herramienta aporta un complemento objetivo a la evaluación clínica habitual y que la presentación gráfica de las series temporales facilita la lectura rápida del desempeño motor intrasesión. Los neurólogos destacaron la utilidad de contar con biomarcadores cuantitativos de tapping, temblor y coordinación para documentar la evolución en estudios longitudinales. Los profesionales de rehabilitación subrayaron que los informes exportables podrían integrarse en la historia clínica como registro objetivo de la sesión de evaluación motora.

A partir de esa devolución se incorporaron mejoras concretas en la versión final de la interfaz. Se priorizó la vista de evolución longitudinal, que permite filtrar sesiones previas de un mismo paciente por tipo de ejercicio y visualizar la tendencia de biomarcadores seleccionados a lo largo del tiempo. Se implementó el modo de comparación entre dos sesiones, con generación de un texto contrastivo a partir de las métricas agregadas. Se ajustó el formato del informe PDF para resaltar las métricas de mayor relevancia clínica según el tipo de ejercicio. Se añadieron mensajes orientativos en la pantalla de carga sobre condiciones recomendables de grabación (iluminación uniforme, mano visible y centrada, tablero de calibración al inicio cuando se requiera escala métrica). Se refinó la organización de las tablas de resultados para separar métricas primarias de indicadores secundarios de calidad de la señal.

Las respuestas de la guía semiestructurada y los comentarios abiertos permitieron identificar patrones recurrentes en las cinco dimensiones evaluadas en cada ronda: legibilidad de los gráficos temporales, utilidad de las métricas agregadas, claridad del informe exportable, fluidez del flujo de carga y pertinencia para el seguimiento longitudinal. En la tabla 4.1 se sintetiza la observación predominante registrada en cada ítem a lo largo del ciclo de iteraciones, como punto de partida para las mejoras de interfaz descritas en el párrafo anterior.

Estas observaciones permitieron ajustar la presentación y priorizar funcionalidades. Aunque no aportan evidencia de validez diagnóstica, confirmaron que el diseño respondía a necesidades reales de profesionales que evalúan el movimiento

en pacientes con Parkinson.

TABLA 4.1. Síntesis cualitativa de la retroalimentación de profesionales de la salud.

Ítem evaluado	Observación predominante
Legibilidad de gráficos de series temporales	Valoración favorable; útil para seguimiento intrasesión y revisión en consulta.
Utilidad de métricas agregadas	Positiva para documentación objetiva; solicitud de mayor contexto clínico en etiquetas.
Claridad del informe PDF exportable	Adecuada tras iteraciones de formato en la tercera ronda.
Flujo de carga de sesiones	Intuitivo para personal no técnico; sugerencia de mensajes preventivos sobre captura.
Utilidad para seguimiento longitudinal	Alta; motivó la vista de evolución y el modo comparación en la versión final.

4.1.2. Criterios de aceptación

Los criterios de aceptación se alinearon con los objetivos específicos del trabajo: ejecución exitosa del flujo completo sobre material de muestra, coherencia de las métricas en escenarios controlados verificados por prueba unitaria y ejecución documentada de la batería automatizada del repositorio. Para la verificación de componentes críticos se interpretó que el criterio quedaba satisfecho cuando la gran mayoría de los casos aprobaba y los fallos u omisiones restantes se clasificaban como deuda técnica acotada, con causa identificada y sin bloquear el flujo principal del pipeline.

No se dispuso de un conjunto etiquetado de referencia clínica con anotación frame a frame para todos los ejercicios. Por ello, la evaluación cuantitativa de este capítulo se limitó a evidencia de ingeniería y no a estimadores de sensibilidad o especificidad diagnóstica frente a escalas como la MDS-UPDRS [6]. La heterogeneidad de condiciones de grabación en estudios de video para Parkinson refuerza que la validación clínica prospectiva quede como trabajo futuro [2, 1].

4.2. Evaluación de la extracción de métricas

La evaluación del cumplimiento funcional se organizó en dos niveles: verificación de las capacidades transversales del sistema y verificación del conjunto de métricas exportadas según el tipo de ejercicio clínico. En ambos casos el criterio fue la existencia de implementación operativa en la línea de comandos y en el servicio de procesamiento de la API, respaldada por pruebas automatizadas cuando correspondía.

4.2.1. Cumplimiento de funcionalidades planificadas

En la tabla 4.2 se resume el contraste entre las funcionalidades priorizadas en la planificación inicial del trabajo y el estado al cierre del mismo. Las capacidades marcadas como parciales conservan implementación base pero requieren endu-recimiento adicional para entornos regulados (por ejemplo, cumplimiento pleno de HIPAA [15]).

TABLA 4.2. Cumplimiento de funcionalidades planificadas frente al estado final de implementación.

Funcionalidad planificada	Prioridad	Estado final
Ingesta y procesamiento de video clínico	P0	Implementada (cargador de video y API de sesiones).
Estimación de pose con modelo preentrenado	P0	Implementada (modelos de mano y cuerpo completos).
Extracción de métricas motoras por ejercicio	P0	Implementada para los cuatro ejercicios del protocolo.
Visualización temporal e informes	P0	Implementada (gráficos web, exportación en CLI y PDF).
Portabilidad local y nube sin cambiar lógica	P0	Implementada (contenedores locales y despliegue GCP PoC).
Documentación técnica de arquitectura y ejecución	P0	Implementada.
Salvaguardas de privacidad y consentimiento	P0	Parcial (clasificación de datos y control de acceso, sin certificación HIPAA).
Exportación estructurada CSV/JSON	P1	Implementada.
Pruebas unitarias de módulos críticos	P1	Implementada (28 archivos unitarios más 1 de integración).
Validación de métricas en casos controlados	P1	Implementada (señales sintéticas en pruebas unitarias).
Interfaz web de carga y resultados	P2	Implementada (aplicación web con autenticación).
Reportes PDF exportables	P2	Implementada.
Resumen simplificado para pacientes	P2	Parcial (portal de paciente con acceso restringido a sus sesiones).

El análisis evidenció que las capacidades de prioridad alta quedaron cubiertas en su mayoría. Las extensiones de prioridad media superaron el alcance mínimo inicial al incorporar comparación de sesiones, borradores asistidos por modelo de lenguaje y despliegue en Google Cloud Platform [3] mediante infraestructura como código. La brecha principal se concentró en requisitos regulatorios de salud, explicitados en la documentación del repositorio como trabajo pendiente para uso clínico productivo.

4.2.2. Cobertura por tipo de ejercicio

Cada variante de evaluación clínica activa un subconjunto distinto de métricas. La tabla 4.3 relaciona el tipo de ejercicio, la señal principal empleada, las magnitudes calculadas y el respaldo de pruebas automatizadas.

TABLA 4.3. Cobertura de implementación y pruebas automatizadas por tipo de ejercicio.

Ejercicio	Señal principal	Métricas calculadas	Pruebas asociadas
Tapping de dedos	Distancia pulgar-índice	Tasa de tapping, intervalo entre taps, decremento de amplitud, fatiga, ángulo de muñeca, sincronía con metrónomo.	6 módulos (amplitud, frecuencia, regularidad, fase estática, audio, ángulo).
Temblor de mano	Trayectorias de índice, medio y anular por lado	Amplitud y frecuencia de temblor por lado (sin biomarcadores de tapping ni regularidad).	2 módulos espectrales genéricos (amplitud y frecuencia; sin prueba dedicada del cuantificador).
Apertura y cierre de manos	Apertura bilateral de ambas manos	Ciclos de apertura, amplitud, frecuencia, regularidad, velocidades de apertura y cierre, asimetría intermano.	4 módulos específicos del ejercicio.
Prueba nariz-dedo	Distancia índice-nariz normalizada	Toques exitosos, ritmo de alcance, amplitud de alcance, ángulos de brazo, estabilidad postural.	2 módulos de métricas y señales.

Para el tapping de dedos, las pruebas sobre fase estática y detección de impulsos auditivos verificaron que el recorte temporal previo al cálculo de taps respetara la segmentación definida en el diseño del pipeline. Para el temblor de mano, la rama de producción emplea un cuantificador dedicado que procesa las trayectorias de los dedos índice, medio y anular por lado, exportando amplitud y frecuencia sin invocar biomarcadores de tapping ni regularidad del movimiento. Las pruebas unitarias de amplitud y frecuencia validan los módulos espectrales subyacentes.

Para la apertura y cierre de manos, las pruebas de alineación con metrónomo validaron la correspondencia entre flancos de apertura o cierre y eventos auditivos, y las de apertura bilateral verificaron las métricas de asimetría entre manos. Para el ejercicio nariz-dedo, las pruebas de señales confirmaron la asignación izquierda y derecha y el respaldo sobre puntos corporales cuando faltaban landmarks de mano.

4.2.3. Resultados de la batería de pruebas automatizadas

Se ejecutó la batería completa de pruebas del repositorio en el entorno de desarrollo documentado (Python 3.12 con dependencias gestionadas mediante uv). La corrida recopiló 115 casos de prueba distribuidos en 29 archivos (28 unitarios y 1 de integración). El resultado fue de 110 casos aprobados (95,7 %), cuatro omitidos y uno fallido.

Los cuatro casos omitidos correspondieron a la prueba de integración del pipeline por línea de comandos: al no disponerse del video de muestra en el entorno de evaluación. El único fallo se registró en el módulo de asignación bilateral de manos para el ejercicio de apertura y cierre. El caso de prueba esperaba asignación izquierda y derecha a partir del metadato de lateralidad del estimador de pose, mientras que la implementación determina el lado por la posición horizontal de la muñeca en el plano de imagen. Se clasificó como deuda técnica menor: el módulo de apertura bilateral opera correctamente en las pruebas específicas del ejercicio, y el fallo aislado no bloquea el resto del pipeline.

En la tabla 4.4 se listan los 29 archivos de prueba agrupados por dominio funcional.

TABLA 4.4. Inventario de la batería de pruebas automatizadas ejecutada al cierre del trabajo.

Dominio	N	Contenido verificado
Métricas espectrales	3	Amplitud, frecuencia y regularidad en señales sintéticas.
Preprocesamiento y calibración	6	Tablero ChArUco, calibración legacy, fase estática, audio, ángulos, escala markerless.
Calibración markerless	3	Región facial, iris y modelo pinhole sin marcador.
Ejercicios clínicos	6	Apertura de manos, alineación metrónomo, nariz-dedo y asignación bilateral.
Seguimiento y exportación	3	Rastreador temporal, escritura CSV/JSON y almacenamiento de sesión.
Capa API y servicios	6	Seguridad, portal de paciente, almacenamiento, ingesta por lotes, borradores e insights.
Integración	2	Cliente de calibración profunda y pipeline completo por CLI.
Total	29	110 aprobados, 1 fallido, 4 omitidos de 115 casos recopilados

El alto porcentaje de aprobación respaldó que las definiciones matemáticas del pipeline fueran reproducibles en código. El fallo aislado en asignación de manos delimitó la confianza en despliegues automatizados hasta alinear ese caso de prueba con la estrategia de asignación implementada.

4.2.4. Evaluación del seguimiento longitudinal

Una de las capacidades priorizadas tras la retroalimentación de profesionales de la salud fue la visualización de la evolución de biomarcadores a lo largo de múltiples sesiones de un mismo paciente. Esta funcionalidad permite al clínico o investigador observar tendencias en el tiempo sin depender de la comparación manual de informes aislados.

La evaluación de esta capacidad se realizó sobre pacientes de prueba con varias sesiones registradas del mismo tipo de ejercicio. Para cada sesión se extrajeron las métricas agregadas de interés clínico y se representaron en gráficos de evolución longitudinal accesibles desde la vista de detalle del paciente. Cada gráfico muestra la serie temporal de un biomarcador seleccionado, con la variabilidad expresada como media más o menos una desviación estándar por fecha de sesión, y un selector inferior que permite ampliar o desplazarse sobre el intervalo temporal.

A continuación se presentan dos ejemplos representativos, correspondientes al ejercicio de tapping de dedos y al de apertura y cierre de manos. Estas visualizaciones complementan la evaluación cuantitativa de la batería de pruebas al demostrar que el sistema persiste y expone correctamente los resultados históricos de un paciente.

Para el tapping de dedos, en la figura 4.1 se muestra la evolución del índice de fatiga, biomarcador adimensional que relaciona la amplitud final con la amplitud inicial en la ventana dinámica del ejercicio. El gráfico abarca siete sesiones distribuidas entre enero y julio de 2026 y evidencia fluctuaciones entre valores cercanos a 0,7 y 1,1, con un máximo en abril. La banda sombreada alrededor de la curva

refleja la dispersión entre sesiones del mismo periodo. Esta vista permitió a los neurólogos consultados contrastar la tendencia de fatiga motora sin reconstruir manualmente los informes de cada sesión.

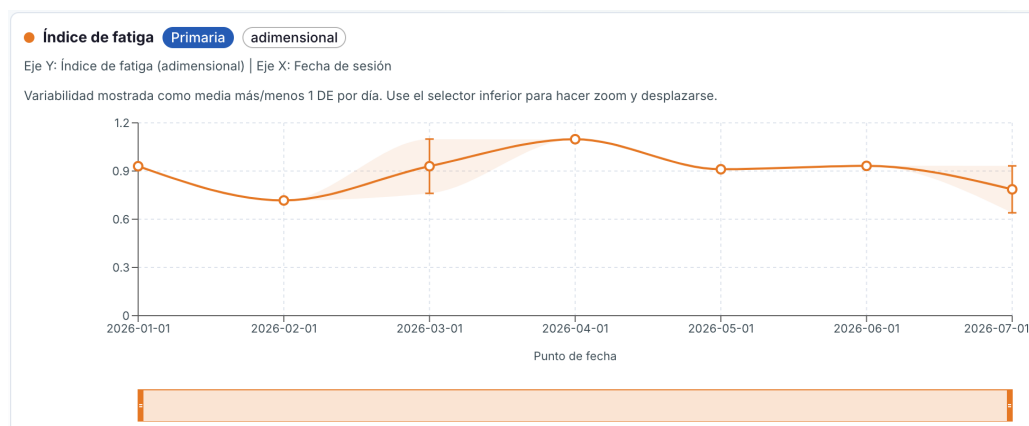


FIGURA 4.1. Evolución longitudinal del índice de fatiga en el ejercicio de tapping de dedos.

En la figura se observa que el índice se mantiene en general por encima de 0,7, lo que sugiere conservación relativa de la amplitud de movimiento a lo largo de la sesión en la mayoría de los registros evaluados. La utilidad verificada aquí no es la interpretación diagnóstica de cada valor aislado, sino la capacidad del sistema de almacenar, recuperar y graficar la serie completa sin pérdida de datos entre sesiones.

Para el ejercicio de apertura y cierre de manos, en la figura 4.2 se presenta la evolución de la velocidad pico de apertura, expresada en milímetros por segundo. En el mismo intervalo temporal, la curva muestra un incremento general desde aproximadamente 20 mm/s en la primera sesión hasta valores cercanos a 60 mm/s en la última, con un descenso intermedio en junio. Los profesionales de rehabilitación señalaron esta magnitud como especialmente informativa para documentar la progresión de la velocidad de apertura en protocolos de seguimiento.

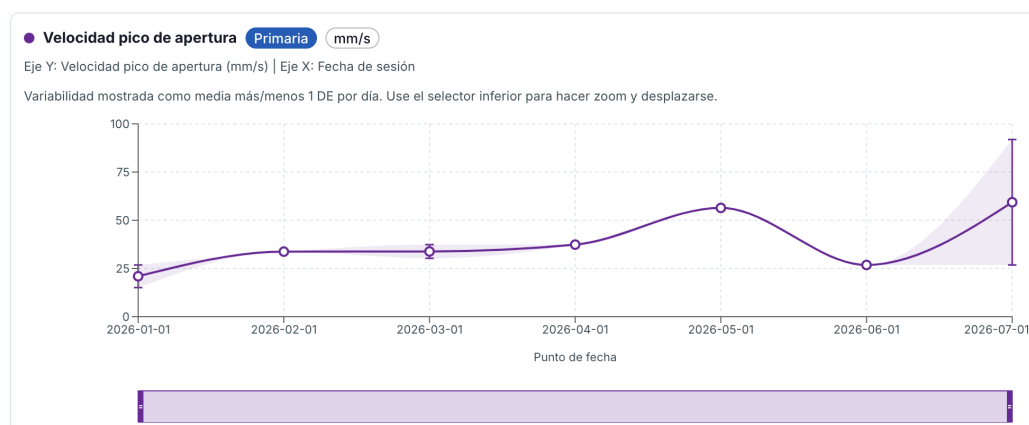


FIGURA 4.2. Evolución longitudinal de la velocidad pico de apertura en el ejercicio de apertura y cierre de manos.

El gráfico confirma que el componente de evolución longitudinal distingue métricas primarias por ejercicio, etiqueta las unidades de medida y representa la

variabilidad asociada a cada punto de fecha. Ambas figuras validan que la funcionalidad solicitada en las rondas de retroalimentación quedó operativa en la versión final de la interfaz y que los datos históricos de un paciente pueden consultarse de forma gráfica para apoyar el seguimiento objetivo del movimiento.

4.3. Análisis de errores

En forma complementaria a la evaluación funcional, se analizaron las situaciones en las que el sistema redujo su fiabilidad operativa durante pruebas manuales y ejecuciones sobre material de muestra. Este estudio no cuantificó tasas de error clínico, sino que clasificó limitaciones de la cadena de procesamiento implementada y sus implicancias para el uso en entorno real.

4.3.1. Familias de degradación

Se identificaron cuatro familias recurrentes de degradación.

La primera corresponde a oclusiones parciales o totales de la mano, que provocaron pérdida de puntos clave en el estimador de pose. Aunque el rastreador temporal rellena breves interrupciones de hasta cinco fotogramas consecutivos, huecos más largos generaron valores faltantes en la señal de distancia pulgar-índice y marcaron métricas como no válidas mediante un indicador de validez en el resultado exportado.

La segunda familia agrupa variaciones bruscas de iluminación y fondos complejos, que redujeron la tasa de detección de la mano en ejercicios de extremidad superior y la tasa de detección por lado en la prueba nariz-dedo. En condiciones de luz lateral intensa o sombras marcadas, la proporción de fotogramas con landmarks válidos descendió de forma notable y las métricas agregadas quedaron condicionadas por la menor densidad de muestras.

La tercera familia se asocia a encuadres subóptimos, en los que la mano ocupaba una fracción reducida del plano de imagen. En esos casos la calibración con tablero ChArUco resultó poco fiable y el sistema recurrió al modo de estimación de escala sin marcador o a parámetros de conversión en píxeles, con mayor incertidumbre en las magnitudes expresadas en milímetros.

La cuarta familia afecta a la prueba nariz-dedo, que depende de la estimación holística de cuerpo completo. Cuando el paciente giraba el torso o una extremidad salía del encuadre, la visibilidad simultánea de nariz, hombros y ambas manos se degradó y disminuyó la densidad de muestras válidas para el cálculo de métricas de alcance y estabilidad.

4.3.2. Casos ilustrativos

En grabaciones con detección exitosa del tablero ChArUco al inicio de la sesión, la conversión a coordenadas métricas en milímetros operó de forma consistente. Cuando el tablero quedó fuera de campo o parcialmente ocluido, el pipeline recurrió al modo markerless, documentado en las pruebas de calibración sin marcador, con mayor incertidumbre en la escala.

En sesiones de tapping de dedos con oclusión transitoria del pulgar, la señal de distancia presentó interpolaciones del rastreador y picos espurios que el módulo

de fase estática no siempre pudo filtrar. Esos artefactos se tradujeron en conteos de taps ligeramente elevados respecto de la inspección visual, sin invalidar la tendencia general de amplitud decreciente que es clínicamente relevante en este ejercicio.

En la prueba nariz-dedo, cuando faltaban landmarks de mano el sistema recurrió a puntos corporales de respaldo, pero con menor densidad de muestras válidas. En sesiones en las que el paciente giraba el torso y una mano salía del encuadre, la tasa de toques exitosos descendía de forma abrupta aunque el movimiento de alcance fuera clínicamente interpretable. Este comportamiento delimita la necesidad de protocolos de captura más estrictos antes de una validación clínica formal.

El propósito de este análisis fue delimitar condiciones de captura recomendables: iluminación uniforme, mano centrada y visible, tablero de calibración al inicio cuando se requiera escala métrica, y encuadre que incluya nariz y hombros en la prueba nariz-dedo. Estas recomendaciones se incorporaron parcialmente en los mensajes orientativos de la interfaz de carga, en respuesta a la retroalimentación de los profesionales consultados.

4.4. Ensayos de integración

Los ensayos de integración verificaron que los componentes del sistema operaran de forma conjunta en los tres modos de ejecución previstos: línea de comandos, contenedores locales y prueba de concepto en la nube.

4.4.1. Pruebas unitarias y de integración del pipeline

Durante el desarrollo se mantuvo activa la batería de pruebas unitarias, que validó módulos aislados mediante señales sintéticas o entradas controladas: calibración con tablero ChArUco, detección de fase estática, impulsos de audio, métricas espectrales, ángulos derivados, apertura de manos, señales de la prueba nariz-dedo y escritura de artefactos de salida. Los resultados agregados de la sección 4.2 constituyeron la evidencia cuantitativa principal de esta capa.

Complementariamente, la prueba de integración del pipeline por línea de comandos ejecuta el flujo completo sobre un video corto y comprueba la generación de los archivos de seguimiento, métricas y resumen de sesión, además de un caso adicional con calibración markerless para el ejercicio de apertura y cierre de manos. Cuando el video de muestra está disponible, esta suite certifica la orquestación global sin intervención de la API.

4.4.2. Despliegue local y en la nube

Se verificó el sistema empaquetado en dos modalidades de ejecución: orquestación local con contenedores y prueba de concepto en Google Cloud Platform.

En el entorno local, el perfil de despliegue coordinó una base de datos relacional, la API con el pipeline embebido, la aplicación web servida por proxy inverso y, en la configuración completa, un servicio auxiliar de calibración profunda. La verificación consistió en la construcción de imágenes, el arranque coordinado de servicios y la ejecución manual de una sesión de análisis desde la interfaz web. Se comprobó la propagación de eventos de progreso en tiempo real, la persistencia de resultados en el directorio de salida montado y la coherencia entre los gráficos

y las tablas de biomarcadores mostrados en pantalla con los valores obtenidos en ejecución directa por línea de comandos. Esta modalidad demostró portabilidad del mismo código fuente respecto del entorno de desarrollo sin contenedores [14].

Para la prueba de concepto en la nube se reutilizó la imagen de contenedor del servicio API, desplegada en computación serverless con base de datos administrada, almacenamiento de objetos para artefactos de sesión, gestión de credenciales y publicación de imagen mediante integración continua. La infraestructura se provisionó con manifiestos de infraestructura como código en un entorno de demostración, con scripts documentados para despliegue y derribo del entorno. La verificación confirmó que el procesador de sesiones de la API ejecutara el mismo pipeline que la línea de comandos local, almacenara salidas en el backend de objetos configurado y sirviera la interfaz empaquetada en el contenedor, sin modificar la lógica de métricas. Las pruebas de almacenamiento respaldaron la abstracción que selecciona backend local o remoto mediante configuración.

En ambos entornos el criterio de aceptación fue la continuidad funcional del flujo analítico extremo a extremo: carga de video, procesamiento, persistencia de resultados y consulta desde la interfaz web. No se realizó una campaña de carga ni medición sistemática de latencia por sesión. El objetivo se limitó a demostrar viabilidad de despliegue y equivalencia operativa entre la ejecución local empaquetada y la nube gestionada [3].

4.4.3. Síntesis de la verificación integral

Las pruebas unitarias, las pruebas de integración del pipeline, el ensayo con contenedores locales y el despliegue de prueba de concepto en la nube cerraron el ciclo de verificación de implementación planteado en los objetivos del trabajo. El sistema procesó videos de ejercicios clínicos, calculó métricas diferenciadas por tipo de ejercicio, exportó resultados estructurados, expuso visualizaciones de evolución longitudinal y permaneció operativo en entornos local y cloud con configuración acotada a variables de entorno y credenciales.

La tabla de inventario de pruebas, las figuras de evolución longitudinal de este capítulo, la retroalimentación positiva de neurólogos y profesionales de rehabilitación, y los ensayos de despliegue integran el cuerpo de resultados que demuestra la efectividad de la implementación al cierre del trabajo.

Capítulo 5

Conclusiones

En este capítulo se presentan las conclusiones del trabajo. Se analiza el grado de cumplimiento de los objetivos enunciados en el capítulo 1, se repasan los conocimientos de la carrera aplicados durante el desarrollo y se proponen líneas de continuidad para el sistema.

5.1. Logros alcanzados

El objetivo general del trabajo se cumplió: se diseñó e implementó un sistema de análisis digital de oscilaciones y desempeño motor que procesa videos de ejercicios clínicos asociados a la enfermedad de Parkinson, deriva métricas cuantitativas mediante estimación automática de pose y procesamiento de señales, y las presenta en visualizaciones orientadas al personal de salud.

Los objetivos específicos del capítulo 1 quedaron cubiertos en lo esencial. Se construyó un flujo completo desde la ingesta de video hasta la exportación de resultados, con calibración espacial, seguimiento temporal y cálculo de métricas diferenciadas según el tipo de ejercicio clínico. Se integraron modelos preentrenados de estimación de pose y se desarrolló la capa de exposición mediante servicios web, interfaz gráfica, informes exportables y vistas de seguimiento longitudinal. La efectividad de la implementación se verificó mediante pruebas automatizadas y ensayos de integración en entornos local y en la nube, con resultados favorables en su conjunto y deuda técnica acotada, según el análisis del capítulo 4.

El sistema superó el alcance mínimo inicial al incorporar comparación entre sesiones, interpretación asistida sobre métricas agregadas y despliegue reproducible en contenedores y en una prueba de concepto en infraestructura gestionada [3, 14]. La retroalimentación de profesionales de la salud, entre neurólogos y kinesiólogos, fue positiva respecto de la utilidad de las métricas y de los informes, y orientó mejoras en la presentación de resultados y en el seguimiento entre sesiones.

El cumplimiento no fue absoluto en todos los frentes planificados. Las salvaguardas de privacidad, el portal de paciente y la cobertura plena de las pruebas de integración con material de video real quedaron en estado parcial o condicionado. Esas brechas se abordan en la sección 5.3.

Frente al estado del arte, el trabajo se inscribió en la línea de plataformas de análisis markerless basadas en visión por computadora, como las revisadas en el capítulo 1 [6, 1, 2]. El aporte diferencial radica en la orientación a servicios, la portabilidad entre entornos de ejecución y el seguimiento longitudinal multiejercicio en una misma herramienta. No corresponde afirmar superioridad diagnóstica: la

evaluación verificó la implementación y no la concordancia con escalas clínicas de referencia, de modo que el sistema debe entenderse como complemento objetivo de la evaluación habitual y no como su reemplazo.

5.2. Conocimientos aplicados

El desarrollo articuló conocimientos de distintas áreas de la carrera. Del aprendizaje automático y la visión por computadora se aplicaron criterios para seleccionar, integrar y orquestar modelos preentrenados, con atención a sus condiciones de validez, su costo computacional y sus limitaciones ante condiciones adversas de captura. La capa interpretativa basada en un modelo de lenguaje exigió además nociones de diseño de instrucciones, control de la generación y resguardo de datos sensibles.

Del procesamiento de señales y la estadística se emplearon técnicas de análisis temporal y espectral, detección de eventos en series cinemáticas, medidas de variabilidad y regularidad, y agregación por sesión para apoyar la lectura clínica de los resultados.

De la ingeniería de software se aplicaron diseño modular, contratos de datos entre etapas, pruebas automatizadas, persistencia relacional, desarrollo de servicios y aplicaciones web, empaquetado con contenedores e infraestructura como código. También se consideraron prácticas de manejo responsable de datos de salud acordes a los requerimientos éticos de la planificación inicial.

5.3. Trabajo futuro

Las líneas de continuidad se agrupan en datos, arquitectura, despliegue y validación clínica.

En materia de datos, la prioridad es conformar un conjunto de grabaciones más amplio y anotado, bajo consentimiento informado y resguardo regulatorio, que habilite estudios de concordancia entre las métricas del sistema y puntuaciones clínicas de referencia [1]. También conviene completar la cobertura de pruebas de integración en el entorno de desarrollo continuo.

En la arquitectura del sistema quedan mejoras identificadas durante los ensayos: cerrar la deuda técnica residual en las pruebas automatizadas, externalizar el estado de progreso de los análisis para habilitar escalado horizontal, medir de forma sistemática el rendimiento por sesión y explorar el uso de información de profundidad para un análisis tridimensional del movimiento [7].

En el plano del despliegue y el cumplimiento normativo, el paso hacia un uso clínico productivo exige endurecer las salvaguardas de privacidad hasta alcanzar conformidad plena con marcos regulatorios aplicables [15]. Finalmente, la validación clínica prospectiva con evaluadores independientes y protocolos de captura estandarizados constituye el paso necesario para transformar el prototipo verificado en una herramienta de apoyo a la decisión con evidencia de validez diagnóstica [2].

Bibliografía

- [1] Krista G. Sibley, Christine Girges, Ehsan Hoque y Thomas Foltynie. «Video-Based Analyses of Parkinson's Disease Severity: A Brief Review». En: *Journal of Parkinson's Disease* 11.Suppl. 1 (2021), S83-S93. DOI: [10.3233 / JPD-202402](https://doi.org/10.3233/JPD-202402).
- [2] Pasquale Maria Pecoraro, Luca Marsili, Alberto J. Espay, Matteo Bologna y Lazzaro di Biase. «Computer Vision Technologies in Movement Disorders: A Systematic Review». En: *Movement Disorders Clinical Practice* 12.9 (2025), págs. 1229-1243. DOI: [10.1002/mdc3.70123](https://doi.org/10.1002/mdc3.70123).
- [3] Google LLC. *Google Cloud Platform*. <https://cloud.google.com/>. Accedido: 2026-05-31. 2026.
- [4] Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei y Yaser Sheikh. «OpenPose: realtime multi-person 2D pose estimation using Part Affinity Fields». En: *IEEE transactions on pattern analysis and machine intelligence* 43.1 (2019), págs. 172-186.
- [5] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg y Matthias Grundmann. «MediaPipe: A Framework for Building Perception Pipelines». En: *arXiv preprint arXiv:1906.08172* (2019).
- [6] Gabriela Acevedo, Florian Lange, Carolina Calonge, Robert Peach, Joshua K. Wong y Diego L. Guarín. «VisionMD: An Open-Source Tool for Video-Based Analysis of Motor Function in Movement Disorders». En: *npj Parkinson's Disease* 11.1 (2025), pág. 27. DOI: [10.1038/s41531-025-00876-6](https://doi.org/10.1038/s41531-025-00876-6).
- [7] Diego L. Guarín. «Video-based quantification of hand postural tremor without external references. Integrating postural tremor quantification into visionMD». En: *npj Parkinson's Disease* 11 (2025), pág. 351. DOI: [10.1038 / s41531-025-01196-5](https://doi.org/10.1038/s41531-025-01196-5).
- [8] Guido Van Rossum y Fred L. Drake. *Python 3 Reference Manual*. CreateSpace. Scotts Valley, CA, 2009.
- [9] Gary Bradski. «The OpenCV Library». En: *Dr. Dobb's Journal of Software Tools* (2000).
- [10] Brian McFee, Colin Raffel, Dawen Liang, Daniel P. W. Ellis, Matt McVicar, Eric Battenberg y Oriol Nieto. «librosa: Audio and Music Signal Analysis in Python». En: *Proceedings of the 14th Python in Science Conference (SciPy)*. 2015, págs. 18-25. DOI: [10.25080/Majora-7b98e3ed-003](https://doi.org/10.25080/Majora-7b98e3ed-003).
- [11] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Wickers, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio

- H. Ribeiro, Fabian Pedregosa y Paul van Mulbregt. «SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python». En: *Nature Methods* 17 (2020), págs. 261-272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [12] Sebastián Ramírez. *FastAPI*. <https://fastapi.tiangolo.com/>. Disponible: 2026-04-15. 2024.
- [13] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. <https://www.postgresql.org/>. Disponible: 2026-04-15. 2024.
- [14] Dirk Merkel. «Docker: Lightweight Linux Containers for Consistent Development and Deployment». En: *Linux Journal* 2014.239 (2014).
- [15] U.S. Department of Health and Human Services. *Health Insurance Portability and Accountability Act of 1996 (HIPAA)*. Pub. L. 104-191, 110 Stat. 1936. 1996.
- [16] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas y M. J. Marín-Jiménez. «Automatic generation and detection of highly reliable fiducial markers under occlusion». En: *Pattern Recognition* 47.6 (2014), págs. 2280-2292. DOI: [10.1016/j.patcog.2014.01.005](https://doi.org/10.1016/j.patcog.2014.01.005).
- [17] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang y Matthias Grundmann. *MediaPipe Hands: On-device Real-time Hand Tracking*. arXiv preprint arXiv:2006.10214. 2020. URL: <https://arxiv.org/abs/2006.10214>.
- [18] Valentin Bazarevsky, Ivan Grishchenko, Karthik Raveendran, Tyler Zhu, Fan Zhang y Matthias Grundmann. *BlazePose: On-device Real-time Body Pose tracking*. arXiv preprint arXiv:2006.10204. 2020. URL: <https://arxiv.org/abs/2006.10204>.
- [19] OpenAI. *OpenAI API Reference*. <https://platform.openai.com/docs/api-reference>. Disponible: 2026-04-15. 2026.
- [20] Oren Ben-Kiki, Clark Evans e Ingy d'Netto. *YAML Ain't Markup Language (YAML) Version 1.2*. <https://yaml.org/spec/1.2.2/>. 2021.